



CS348 INFORMATION SYSTEMS

Hisham Benotman

Most slides are adopted with permission from A. Alawini, D. Maier, L. Delacambre, S. Davidson, and A. Parameswaran



Department of Computer Science

Overview of Database Design

- Data requirements collection (e.g., using use cases)
- Conceptual design: (ER Model is used for this.)
 - What are the **entities** and **relationships** we need?
- Logical design:
 - Transform ER design to Relational Schema
- Schema Refinement: (Normalization)
 - Check relational schema for redundancies and related anomalies.
- Physical Database Design and Tuning:
 - Consider typical workloads; (sometimes) modify the database design; **select indexes**.

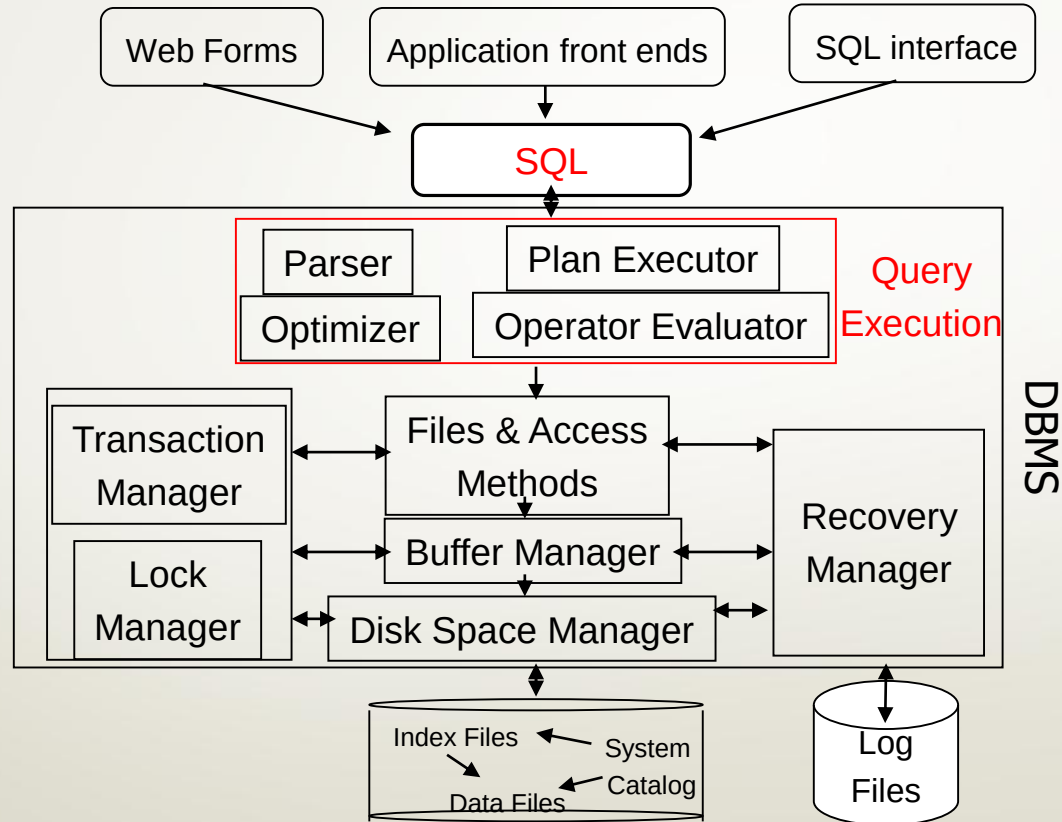


After DB design (and other parts of app. design) is complete, create the database and build the application (or use database for other purposes, such as data analysis).

Two Perspectives of DBMS

- USER PERSPECTIVE
 - **how to use a database system?**
 - conceptual data modeling, the relational and other data models, database schema design, relational algebra, SQL and No-SQL query languages.
- SYSTEMS PERSPECTIVE
 - **how to design and implement a database system?**
 - data representation, indexing, query optimization and processing, transaction processing, and concurrency control.

DBMS Architecture



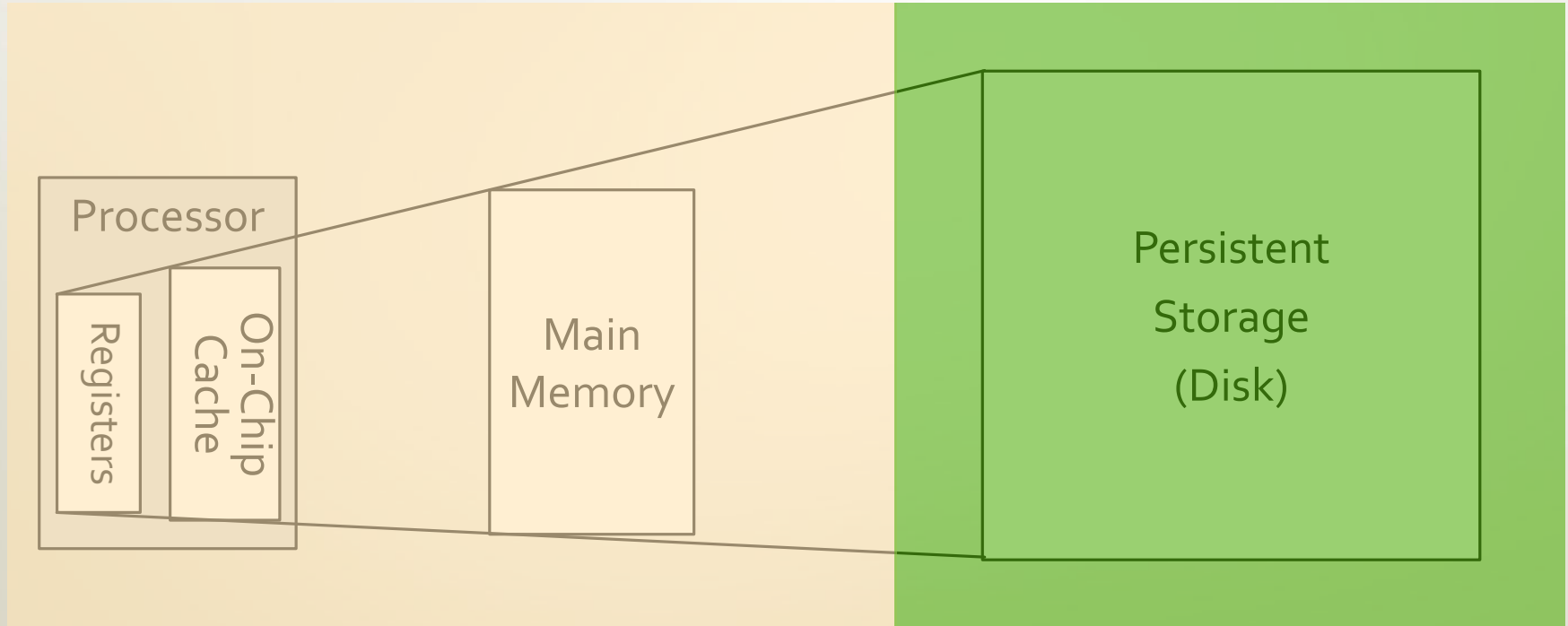
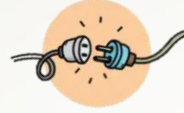
Learning Objectives

- disk and memory
- know how relations are stored on disk
- know how index structures speed up data access
- know the basics of B+ trees
- know the basics of hash indexes
- pick the right indexes for your queries

Outline

- Storage (disk and memory)
- Indexing
 - What is an index? Why do we need it?
 - B+ Trees
 - Hash tables

Simplified Computer Architecture



Speed (ns)

10 ns

10^2 ns

10^7 ns

Size

KB

GB

TB

Cost

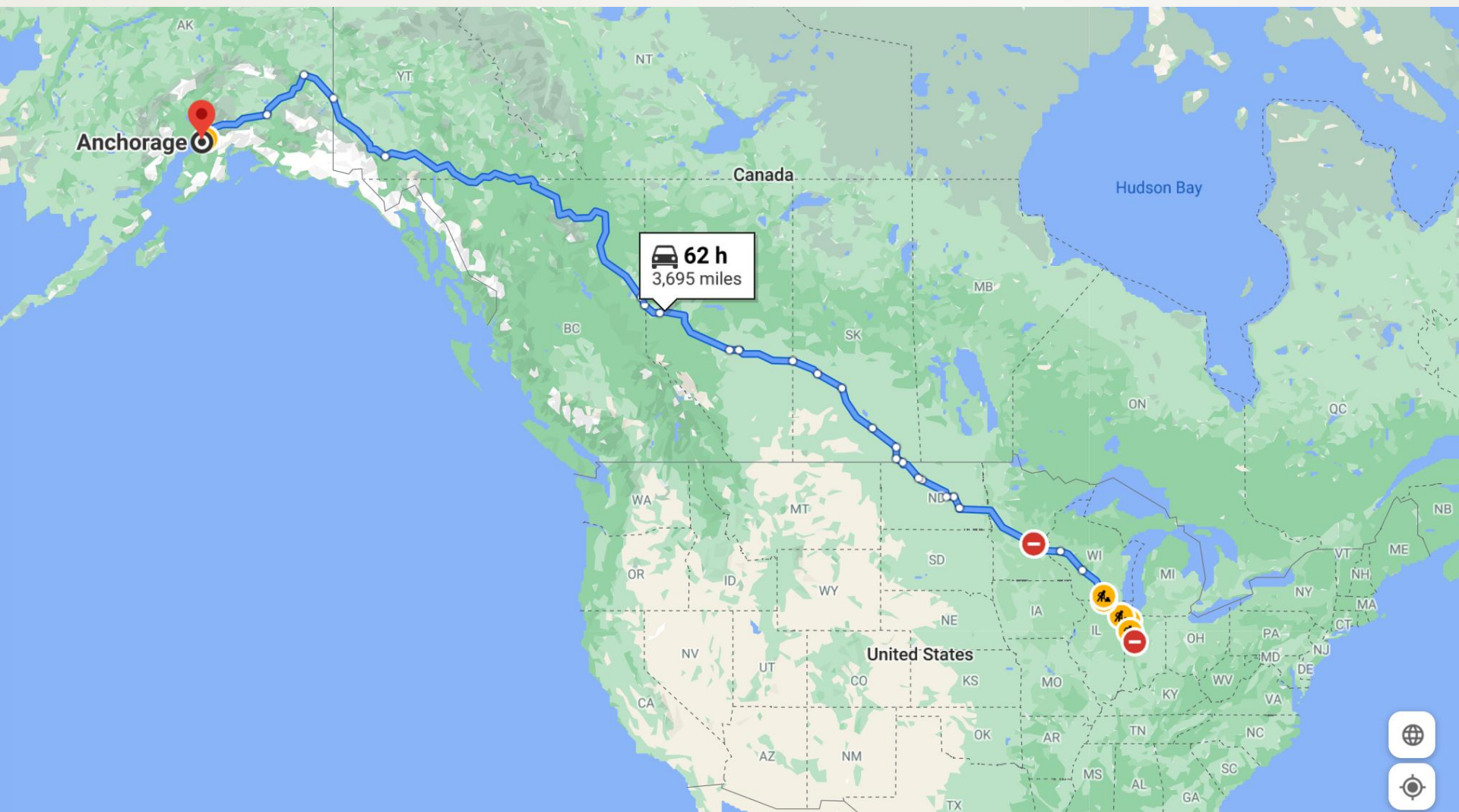
Very expensive

expensive

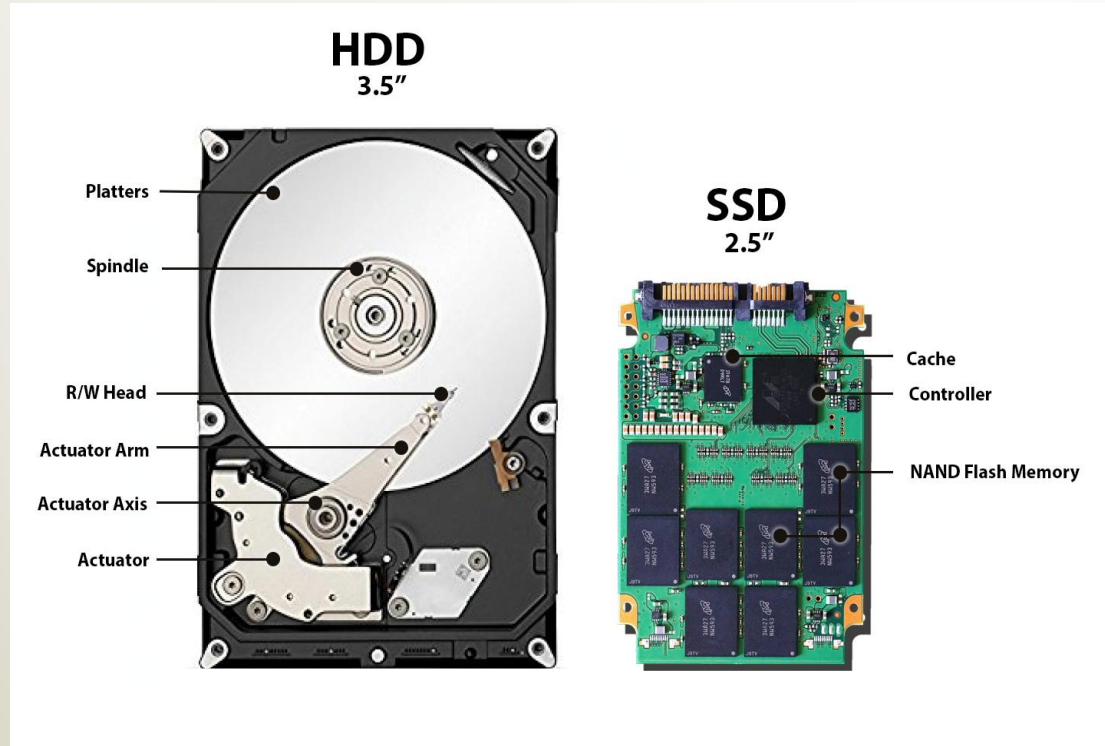
cheap

If accessing memory is similar to looking for something in your room (e.g., a friend's phone number).

Then, accessing disk is like **walking** to Alaska to get the phone number and coming back.



Cost of Accessing Data on Disk



Speed

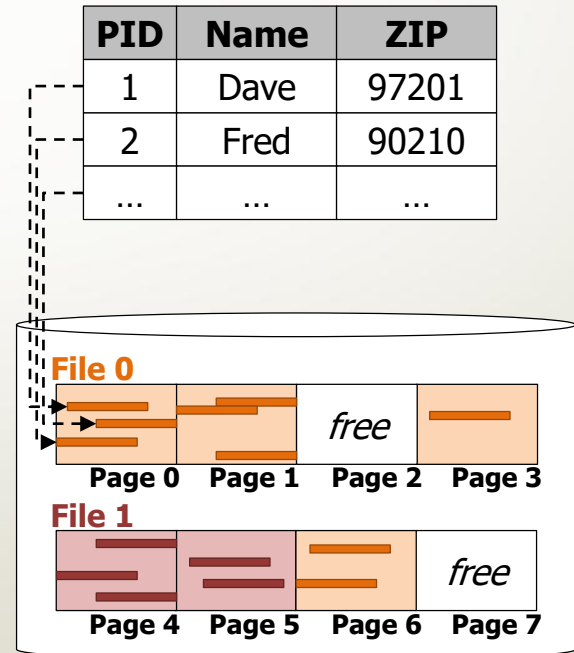
10^7 ns

10^5 ns

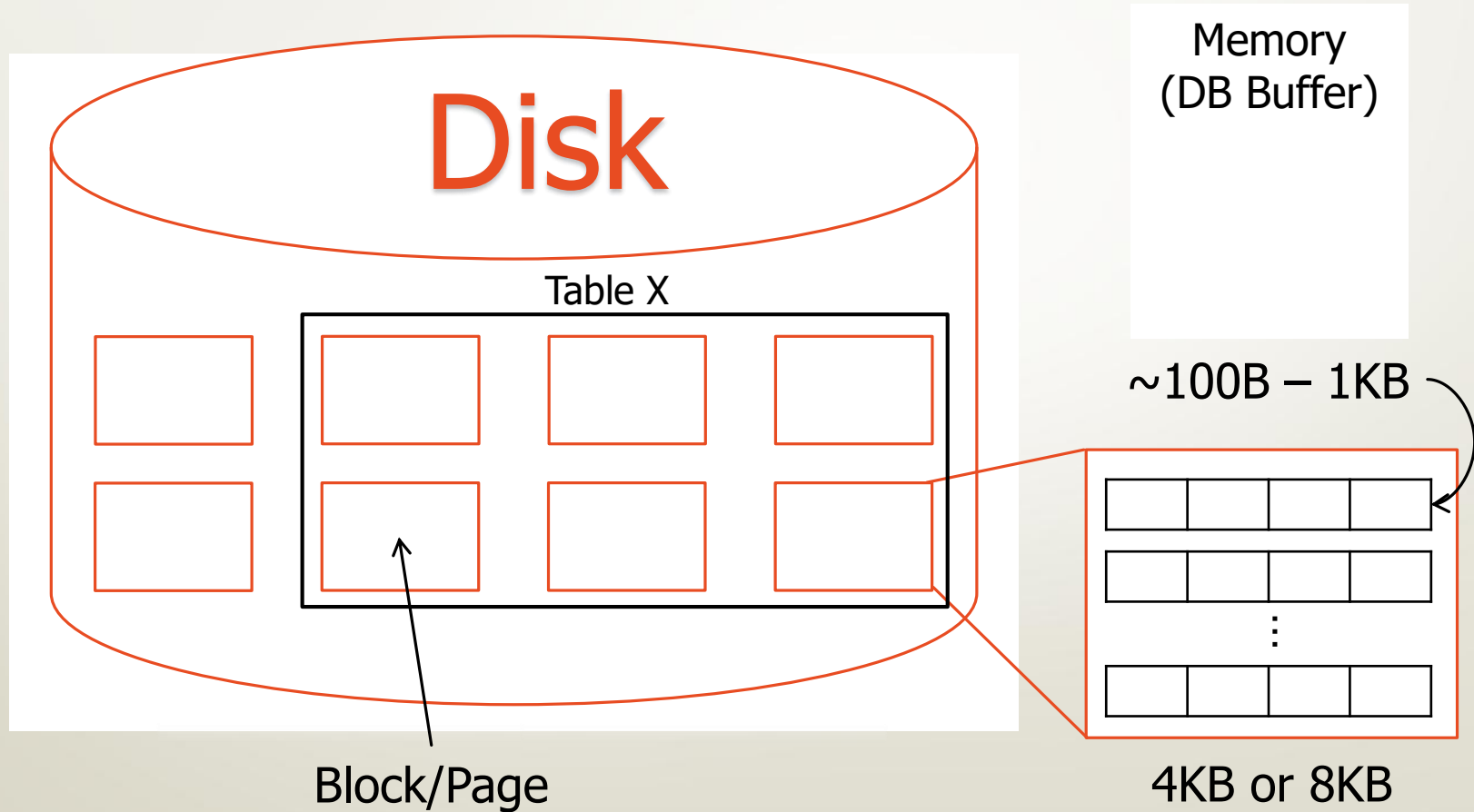
10^2 ns

Conceptual vs. Physical

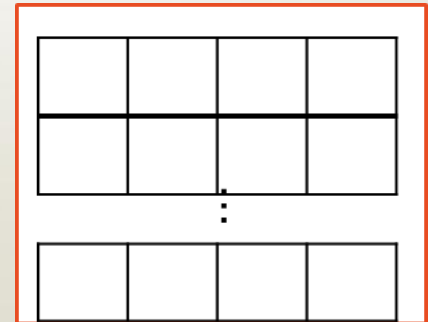
- On the conceptual level, a **relational** DBMS manages **tables of rows and indexes**
- On the physical level, these data structures are implemented as **files of records**
 - file: **one or more pages (blocks)**
 - page: **one or more records**
 - record: **one row**



Disk, Memory, Block, and Record (Row)



Why block



OK. So how do we do simple operations?

Lookups, updates, insertions, and deletions

We have to first bring data to memory.

Bring whole block(s)

If data is changed (block is updated), we need to write the block back to disk (for persistence).

In this lecture, we focus on look ups (i.e.,
SELECT queries)

Outline

- ✓ Storage
- Indexing
 - What is an index? Why do we need it?
- B+ Tree Indexes
 - Basics and Searching
- Hash Indexes

What is the problem?

One catch!



Looking for a keyword in a book



Solution: Index

Search key (keyword)

location

INDEX	
ABC, 164, 321 _n	Anello, Douglas, 60
academic journals, 262, 280–82	animated cartoons, 21–24
Adobe eBook Reader, 148–53	antiretroviral drugs, 257–61
advertising, 36, 45–46, 127, 145–46, 167–68, 321 _n	Apple Corporation, 203, 264, 302
Africa, medications for HIV patients in, 257–61	architecture, constraint effected through, 122, 123, 124, 318 _n
Agee, Michael, 223–24, 225	archive.org, 112
agricultural patents, 313 _n	<i>see also</i> Internet Archive
Aibo robotic dog, 153–55, 156, 157, 160	archives, digital, 108–15, 173, 222, 226–27
AIDS medications, 257–60	Aristotle, 150
air traffic, land ownership vs., 1–3	Armstrong, Edwin Howard, 3–6, 184, 196
Akerlof, George, 232	Arrow, Kenneth, 232
Alben, Alex, 100–104, 105, 198–99, 295, 317 _n	art, underground, 186
alcohol prohibition, 200	artists:
<i>Alice's Adventures in Wonderland</i> (Carroll), 152–53	publicity rights on images of, 317 _n
	recording industry payments to, 52, 58–59, 74, 195, 196–97, 199, 301, 329 _n –30 _n

Looking for rows in a table

```
SELECT * FROM Student  
WHERE ResidenceState = 'Hawaii'
```

Hawaii students can be scattered all around the data file

We have to scan (read) all pages in the student table.

Large number of disk access requests (page IOs)

→ can be a slow query if the table is large



What is the solution?

Indexes

From Minutes to Milliseconds.

Imagine a tool that can reduce the time to compute a query
from minutes to milliseconds.

That is what an index can do for us.

Coming Soon: Question

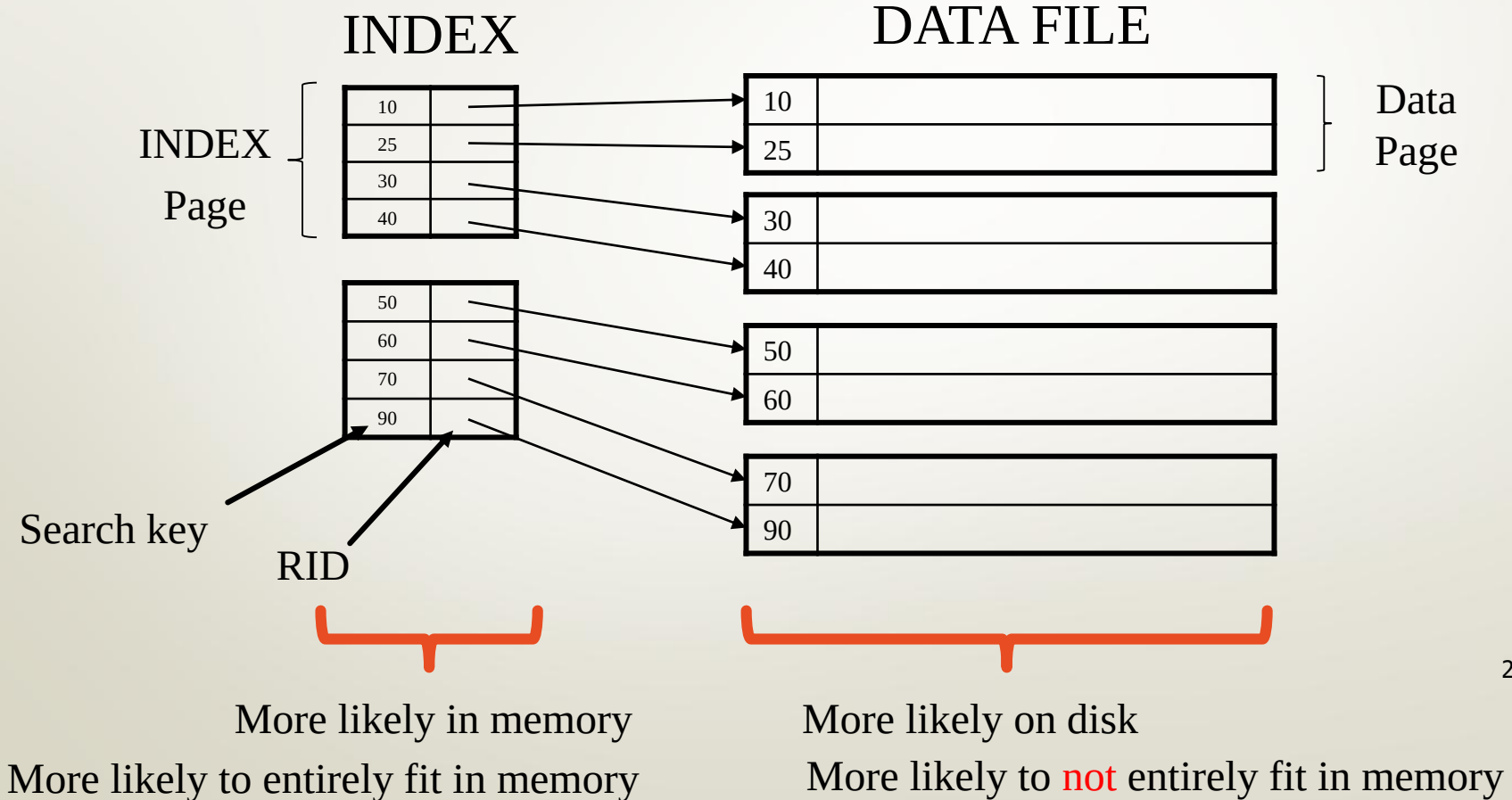
- Why an index is more likely to fit in memory than the data file?
 - Because # of data items in an index entry is likely $\ll$ # of attributes in a row
 - Because the index has a fewer pages than the data file
 - All of the above

Indexes in databases

- An index speeds up selections on the *search key field(s)*
- Search key = any subset of the fields of a relation
 - *Search key* is *not* necessarily the same as a *key or primary key*
- Entries in an index: (k, r), where:
 - k = the search key
 - r = record id (or other information)
 - Record id can be the pair (page #, record # in the page)

Data page size =
index page size

Index



Index

Employee Data File

10		}	Page
25			
30			
40			
50			
60			
70			
90			

SELECT * FROM Student **WHERE Age = 70**

Assuming no index exists, how many page IOs do we need? **four page IOs**

Index

INDEX on Age

10	
25	
30	
40	
50	
60	
70	
90	

Employee Data File

10	
25	
30	
40	
50	
60	
70	
90	

SELECT * FROM Student **WHERE Age = 70**

Assuming index is kept in memory, how many page IOs do we need? **Only one page IO**
(index is in memory, only need to read the last page in the data file from disk)

Some terminology & summary so far

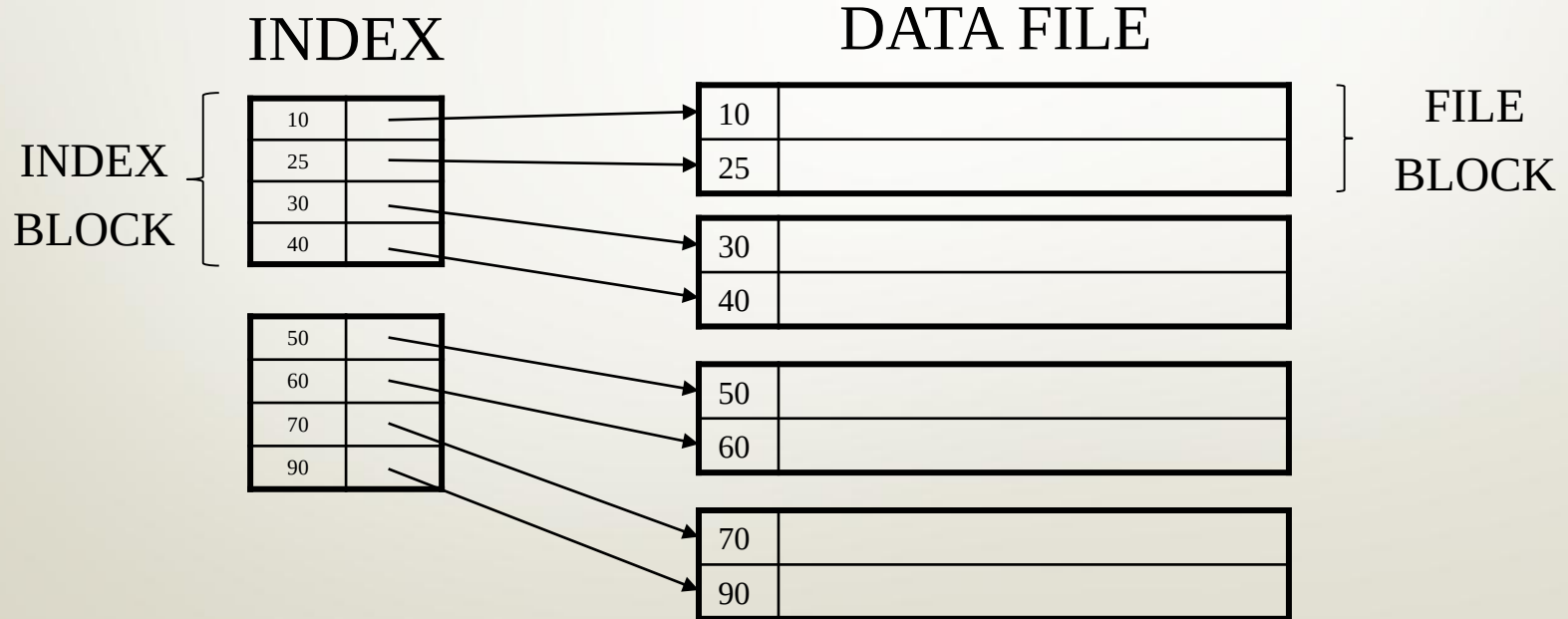
- ***Data file***: has the data corresponding to a relation
- ***Index file***: has the index
- File consists of smaller units called **blocks or pages** (e.g. of size 4 KB or 8 KB)
- # index blocks < # data blocks.
Index can more easily fit into main memory (compared to data file).

Characteristics of Indexes

- Clustered/unclustered
 - Clustered: data records sorted in the search key order
 - Unclustered: data records are NOT sorted in the search key order
- Dense/sparse
 - Dense = each record has an entry in the index
 - Sparse = only some records have
- Primary/secondary
 - Primary = on the primary key
 - Secondary = on any attribute

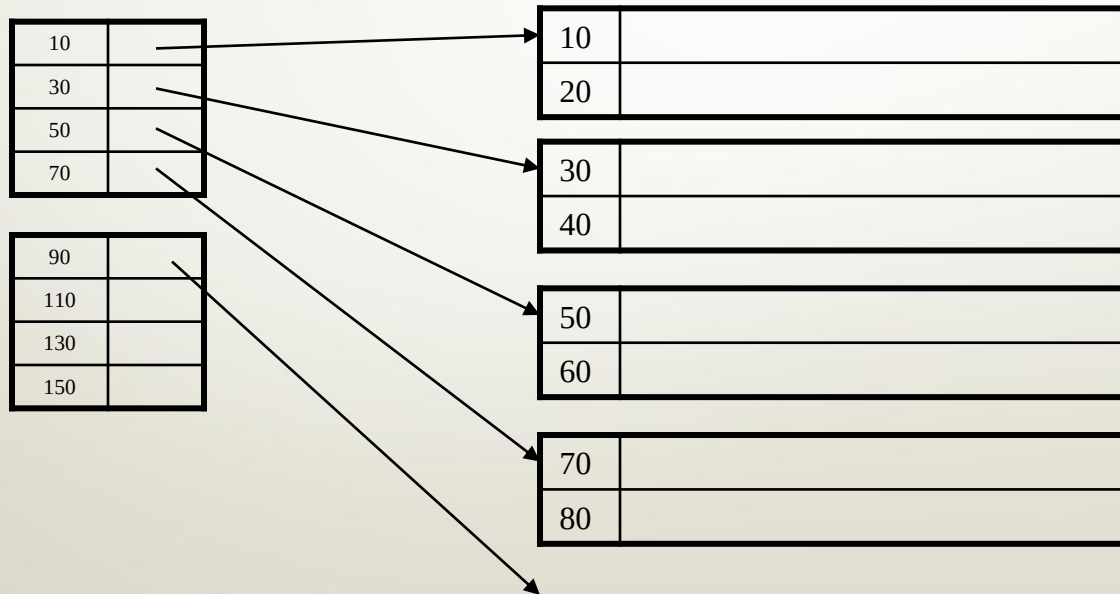
Ex: Clustered, Dense Index

- Clustered: Data file is sorted on the index attribute
- Dense: each record has an entry in the index



Clustered, Sparse Index

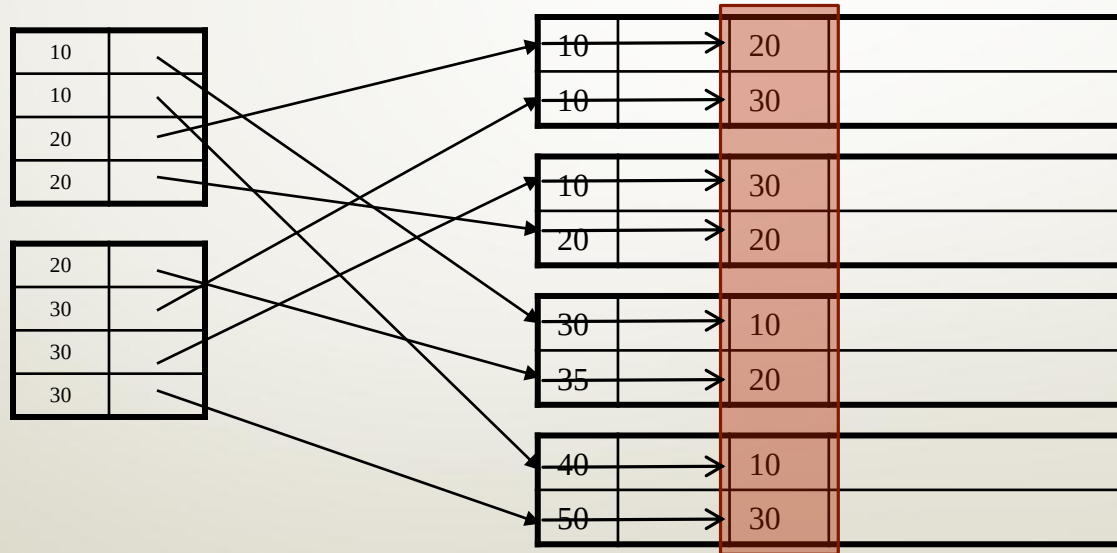
- Sparse index: one key per data block, corresponding to the lowest (or highest) search key in that block



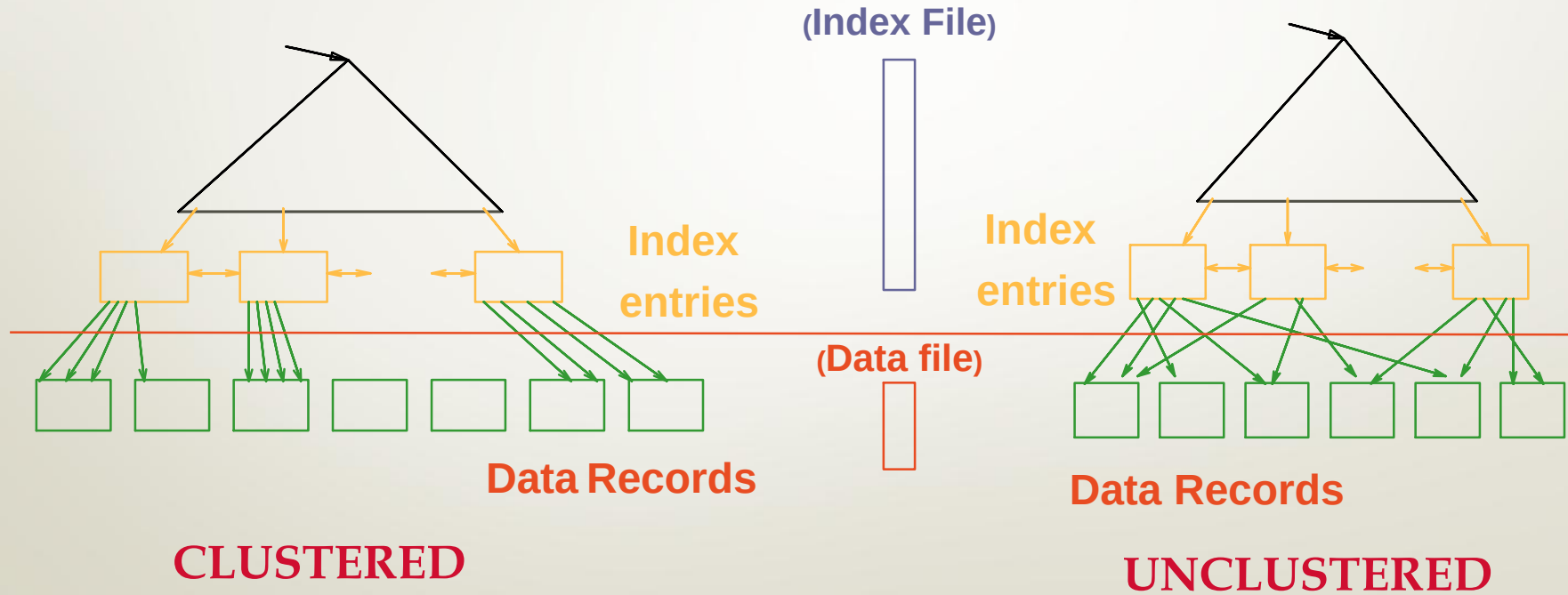
Unclustered Indexes

- Often for indexing other attributes than primary key
- Can it be sparse? **No.**

Secondary



Summary: Clustered vs. Unclustered Index



Clustered/Unclustered Dense/Sparse

	<u>D</u> ense	<u>S</u> parse
<u>C</u> lustered		
<u>U</u> nclustered		

Outline

- ✓ Storage
- ✓ Indexing
 - ✓ What is an index? Why do we need it?
- B+ Tree Indexes
 - Basics and Searching
- Hash Indexes

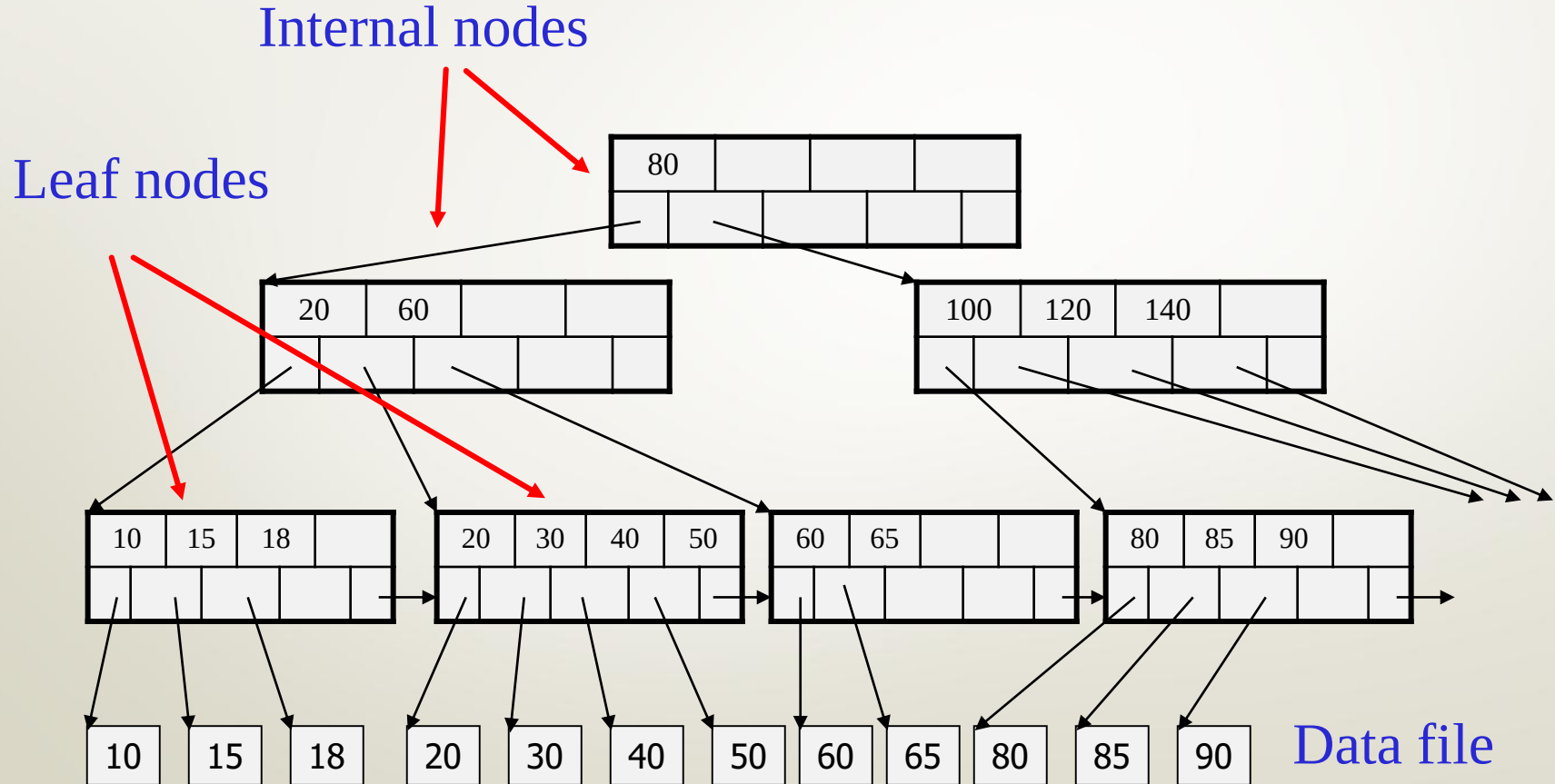
B+ Trees

- Intuition:
 - The index can be very large.
 - Index of index?
 - Index of index of index?
 - How best to create such a multi-level index?

- B+ trees:

Focus on the dense version:
applies to clustered and unclustered settings

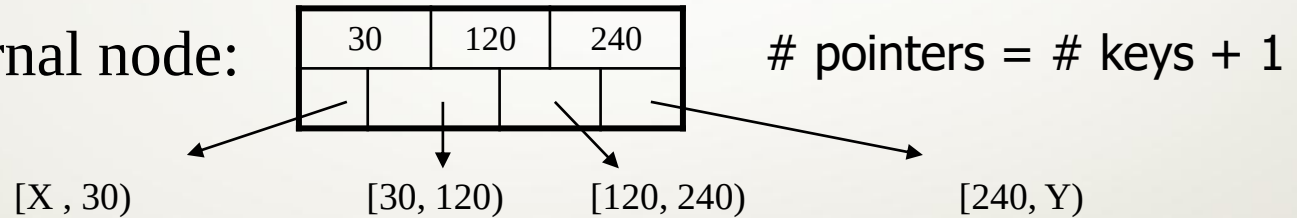
B+ Tree Example



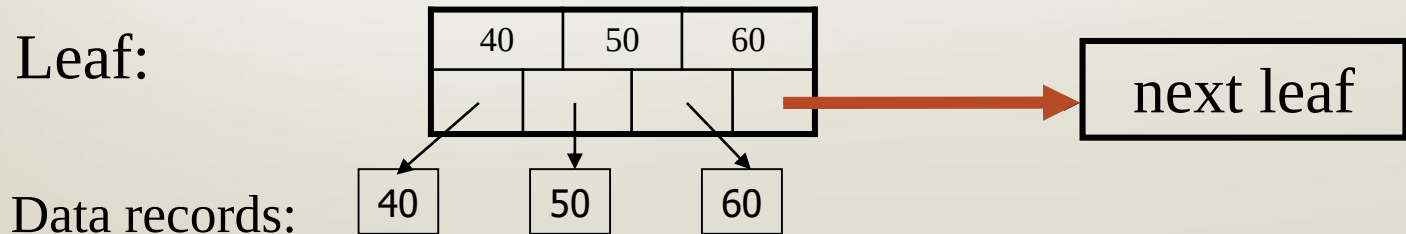
B+ Trees Basics

- B+ Trees are trees with nodes:
Nodes have search keys and pointers to:
 - Other nodes [if the node is an internal node]
 - Data Records [if the node is a leaf]

- Internal node:

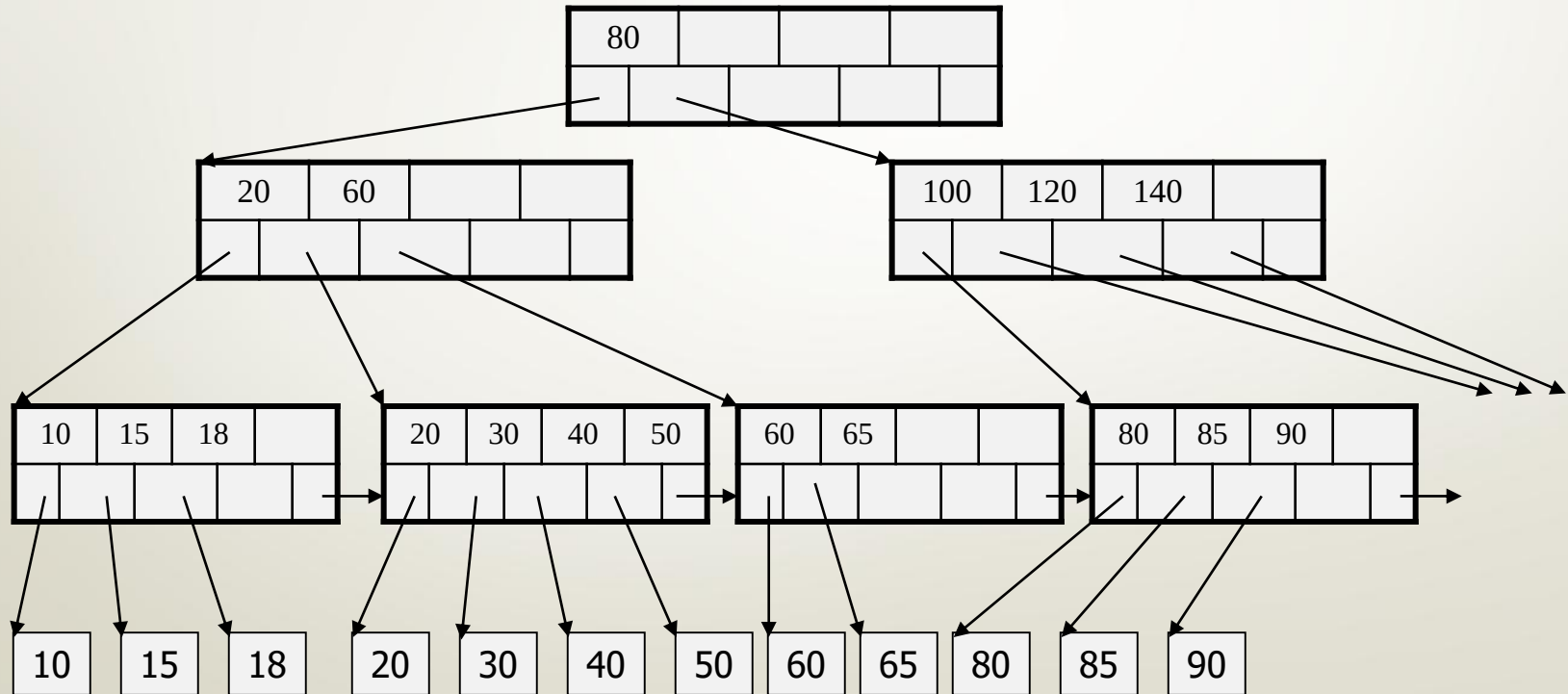


- Leaf:



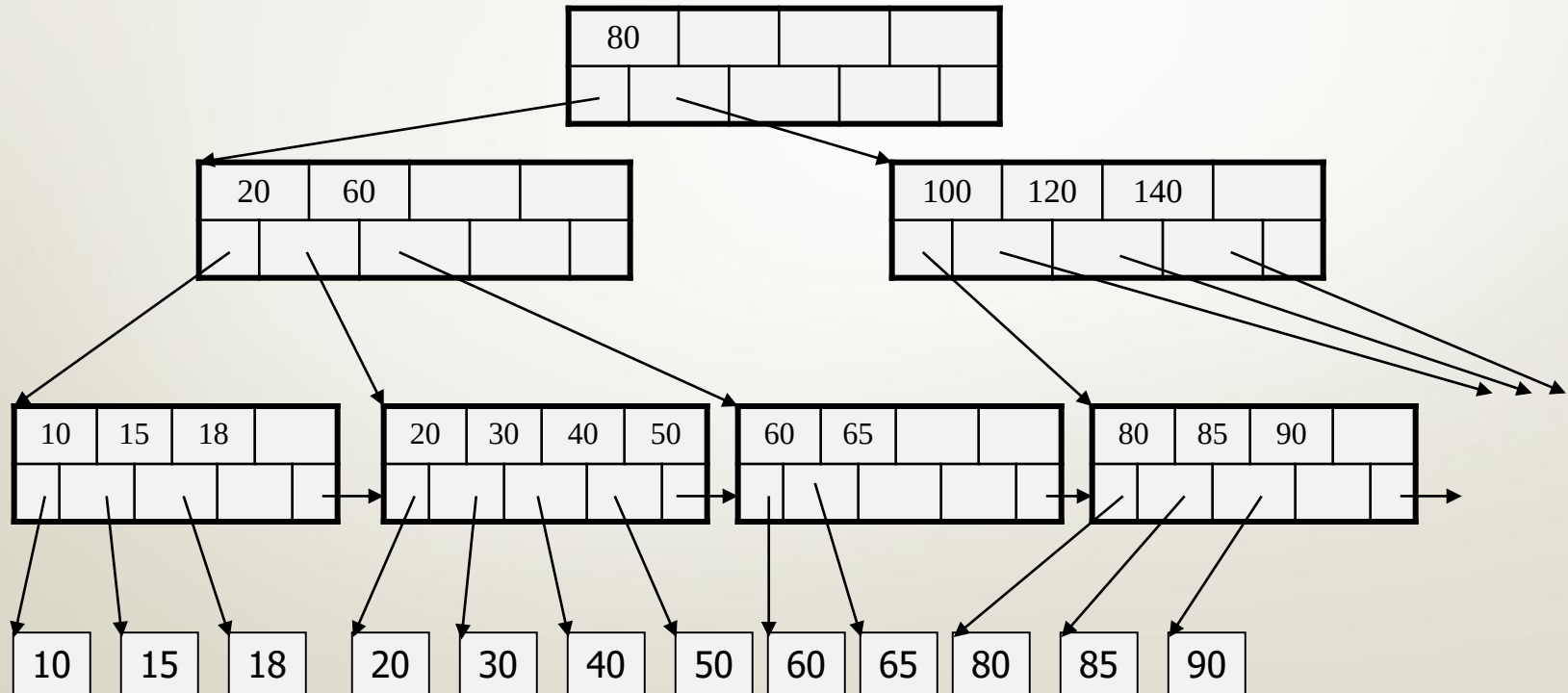
B+ Tree Example

SELECT * FROM Employee WHERE
ID = 80;



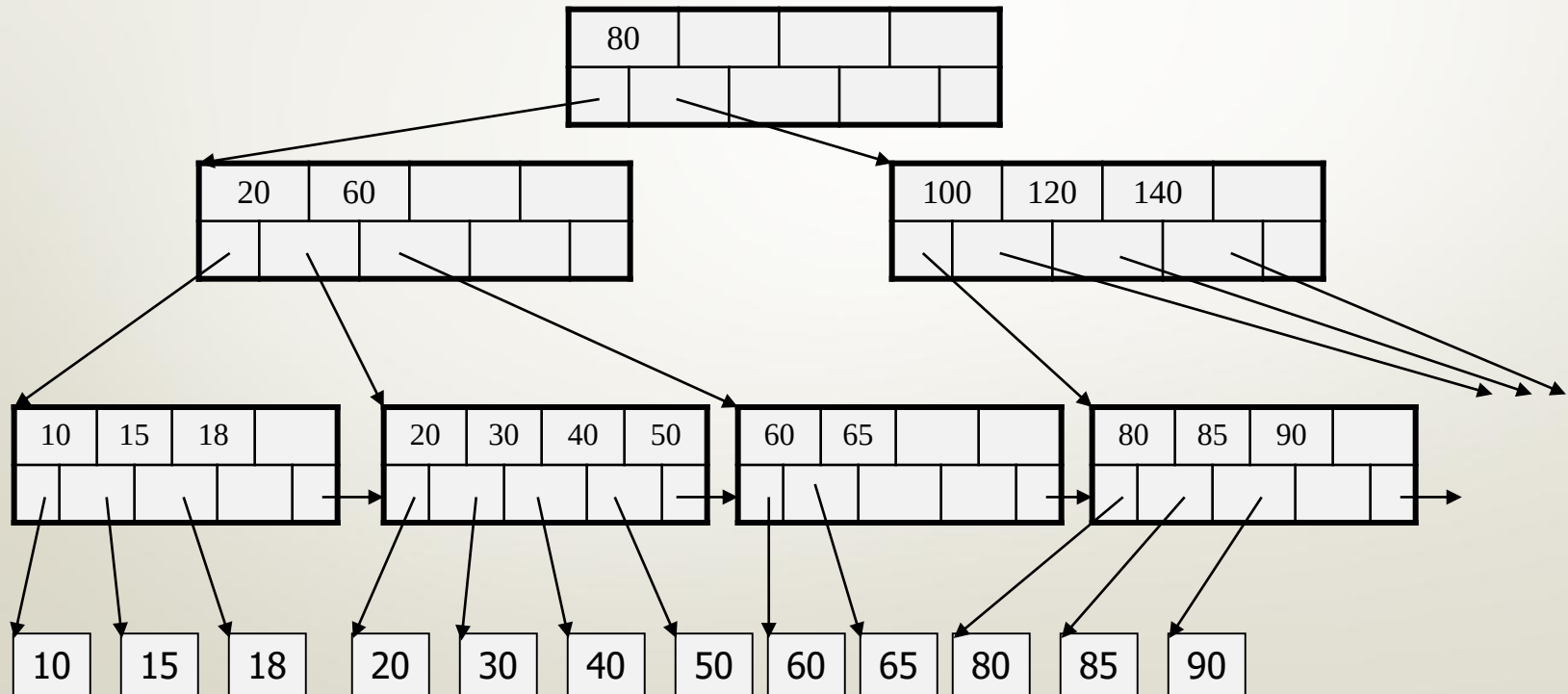
B+ Tree Example

SELECT * FROM Employee WHERE
ID = 82;



B+ Tree Example

SELECT * FROM Employee WHERE
ID $\geq$ 15 AND ID $<$ 65 ;



B+ Trees Basics

- Parameter d = the degree ; n = max keys
- Index nodes are at least half full at all times
 - d is the minimum number of search keys per node (except the root).
- When n is even [*this is our focus for simplicity*]
 - each node has $[d, 2d]$ keys (except root); $n = 2d$

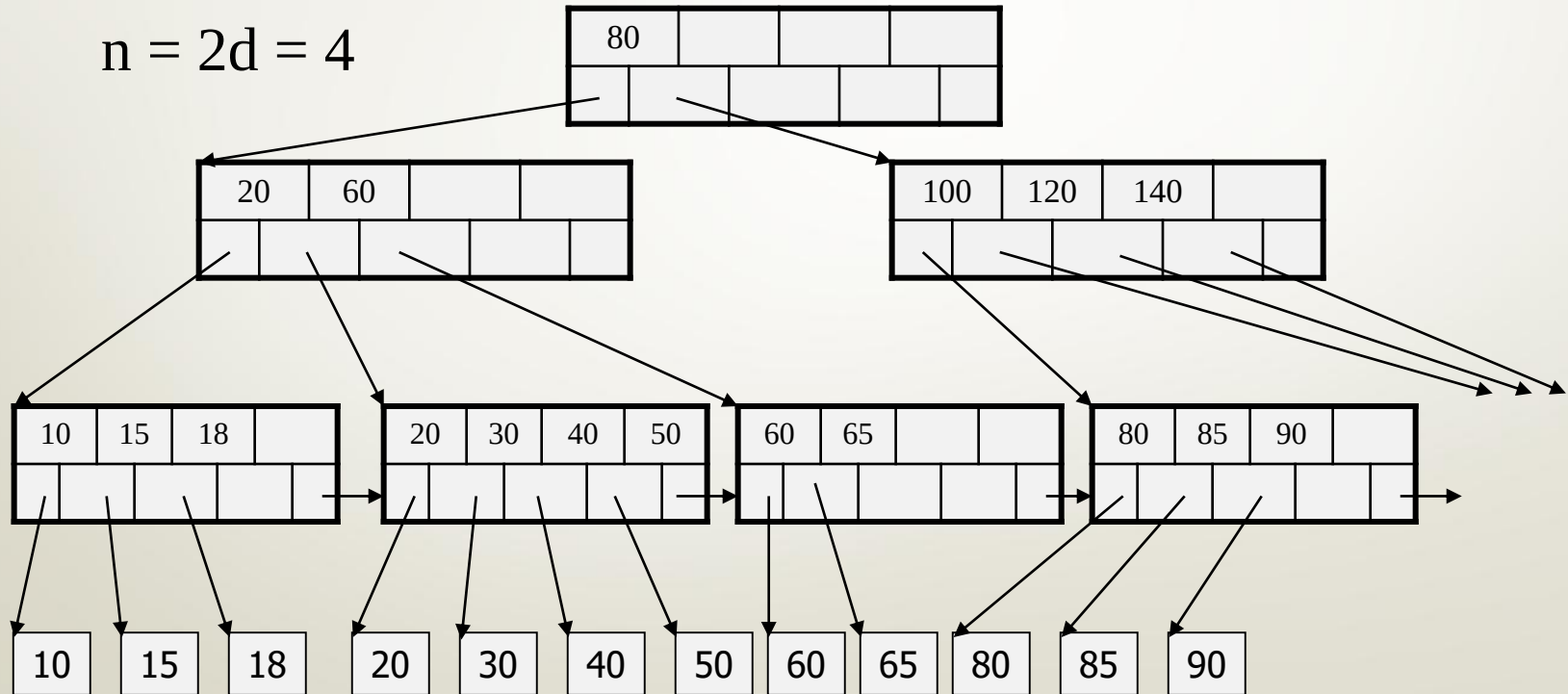
B+ Tree Example

Root can have 1 or more filled in keys

Rest have at least 2 and at most 4

$$d = 2$$

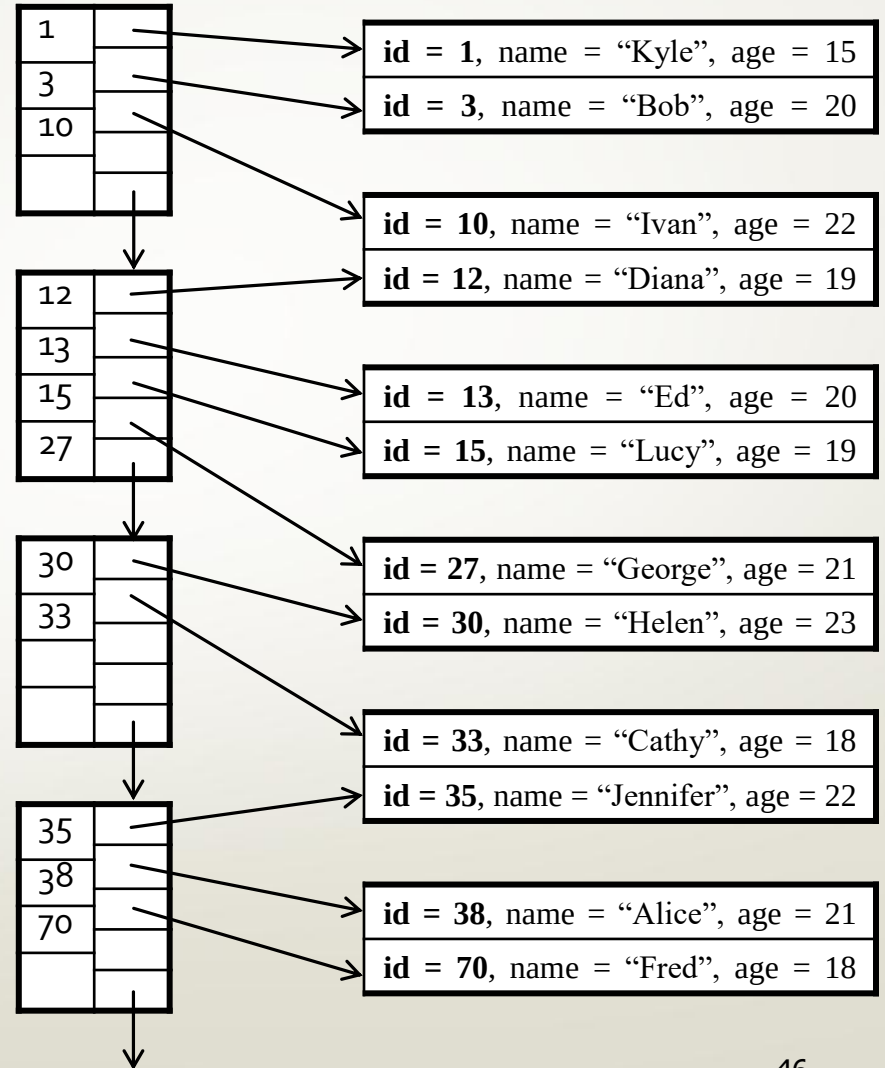
$$n = 2d = 4$$



Clustered dense (entry for every record)

$d = 2; n = 4$

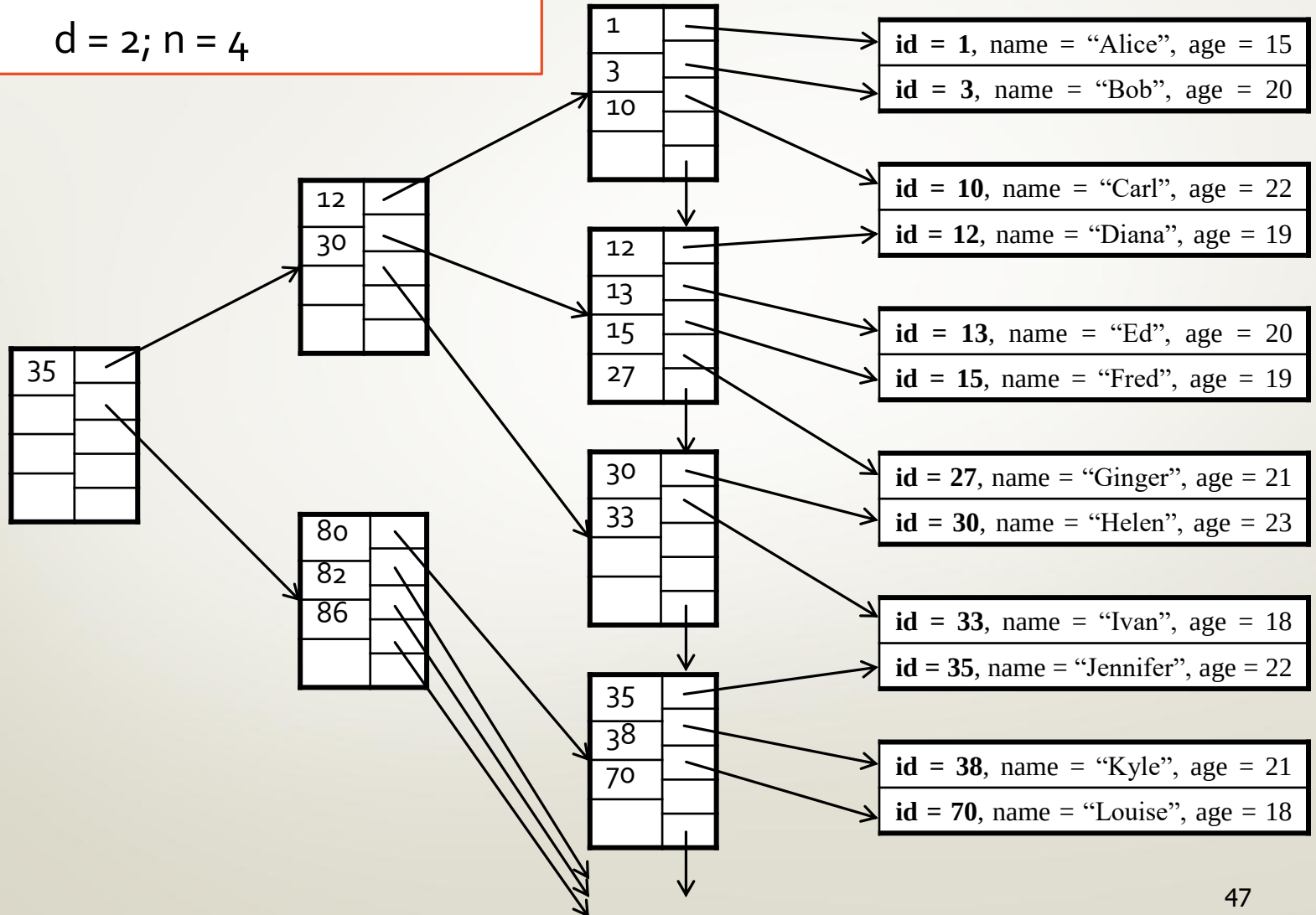
B+ Tree search key = **id**



Clustered dense (entry for every record)

$d = 2; n = 4$

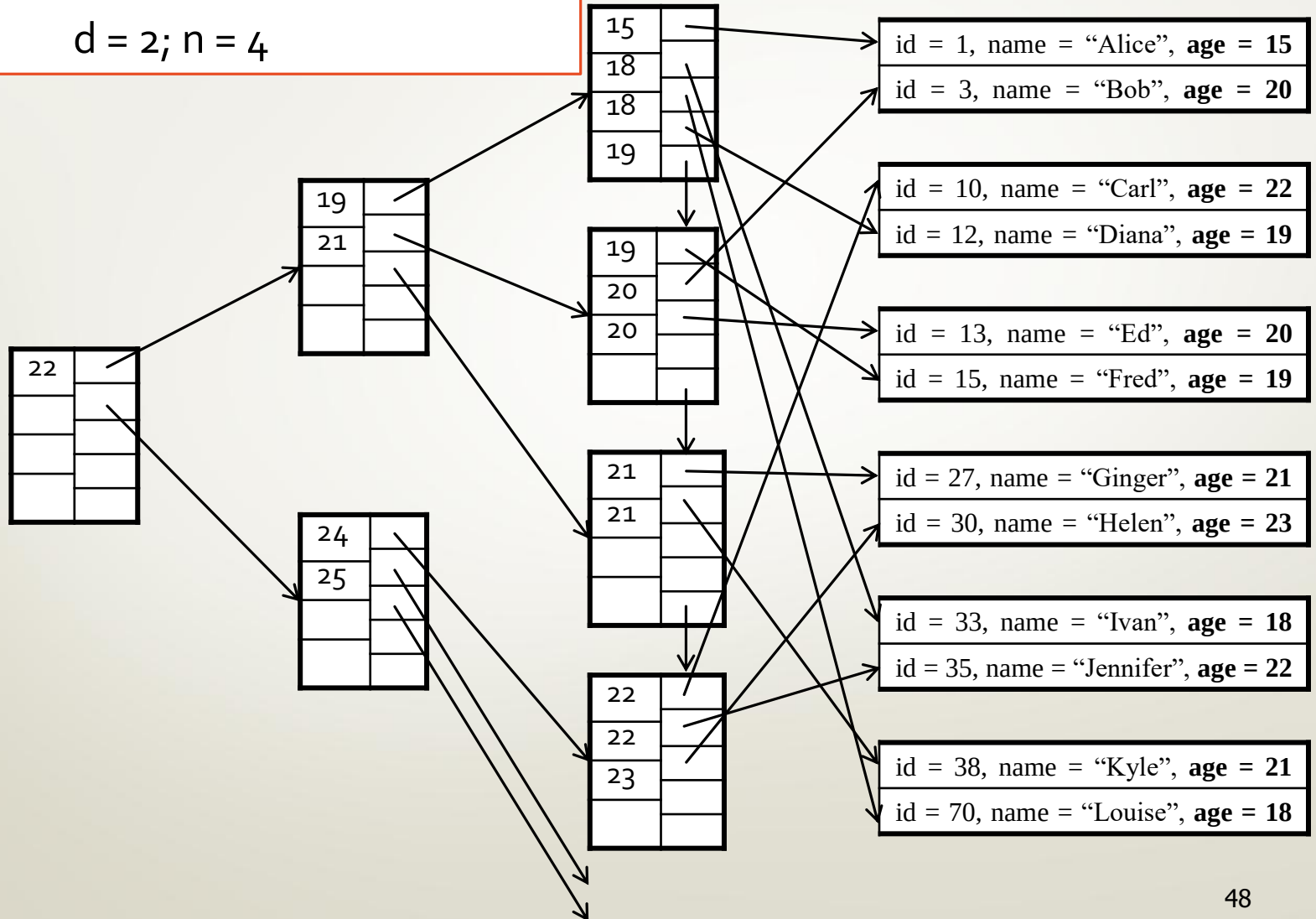
B+ Tree search key = **id**



Unclustered dense (entry for every record)

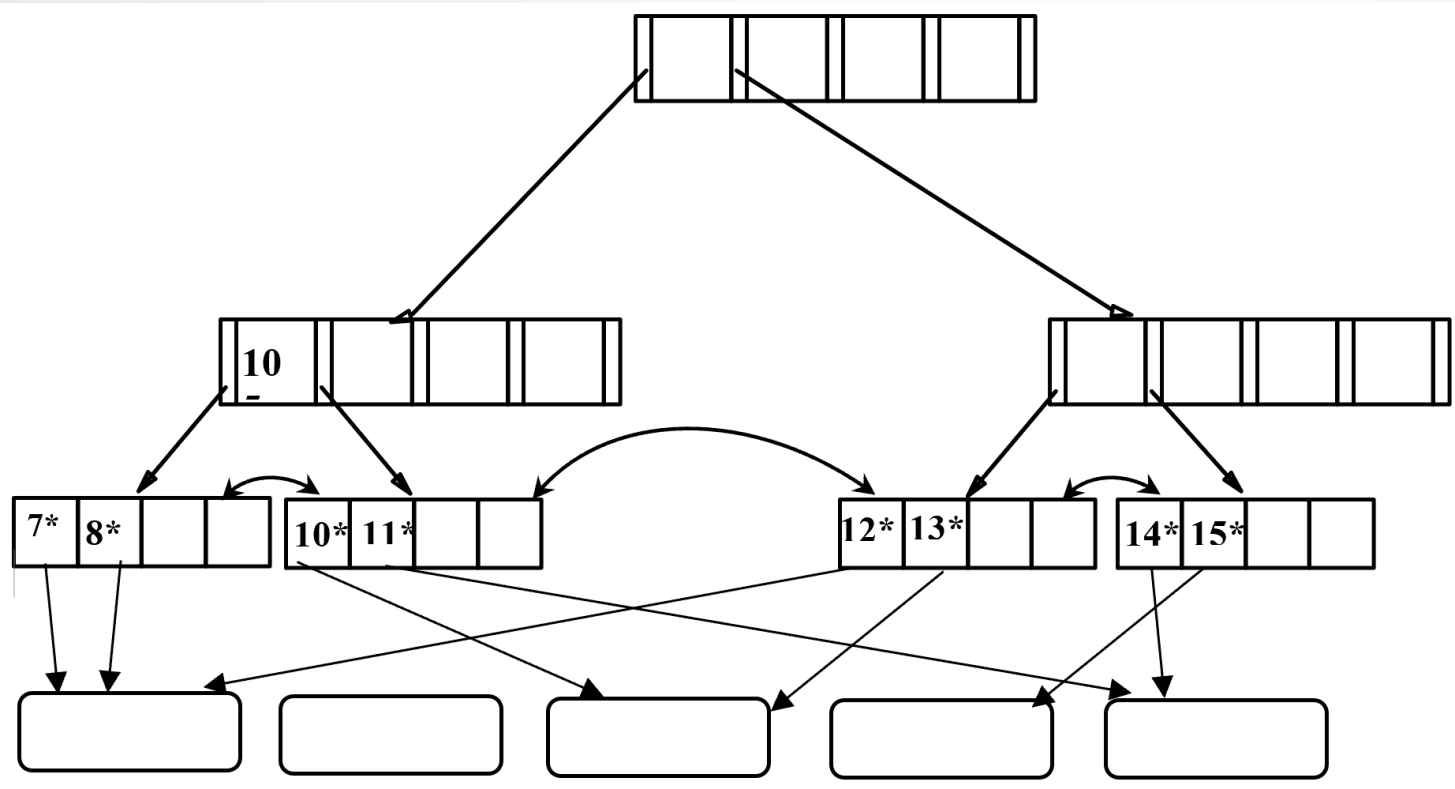
$d = 2; n = 4$

B+ Tree search key = **age**



Exercise

- Consider the following B+tree on customers' shoe size (assuming shoe size is unique).



Exercise

1. Write the missing search keys for the root and the right intermediate node.
2. Assuming the top 2 levels of the index are kept in memory (leaf nodes are in disk), how many page IOs are required to compute the following queries:
 - `Select * FROM Customers WHERE shoe-size = 14.`
 - `Select * FROM Customers WHERE shoe-size >= 8 and shoe-size <= 12.`
3. If we have no index on shoe-size, how many page IOs are required to compute each query?
4. Did the index help in computing both queries?

B+ Trees Basics

- Parameter d = the degree ; n = max keys
- Index nodes are at least half full at all times
 - d is the minimum number of search keys per node (except the root).
- When n is even [*this is our focus for simplicity*]
 - each node has $[d, 2d]$ keys (except root); $n = 2d$

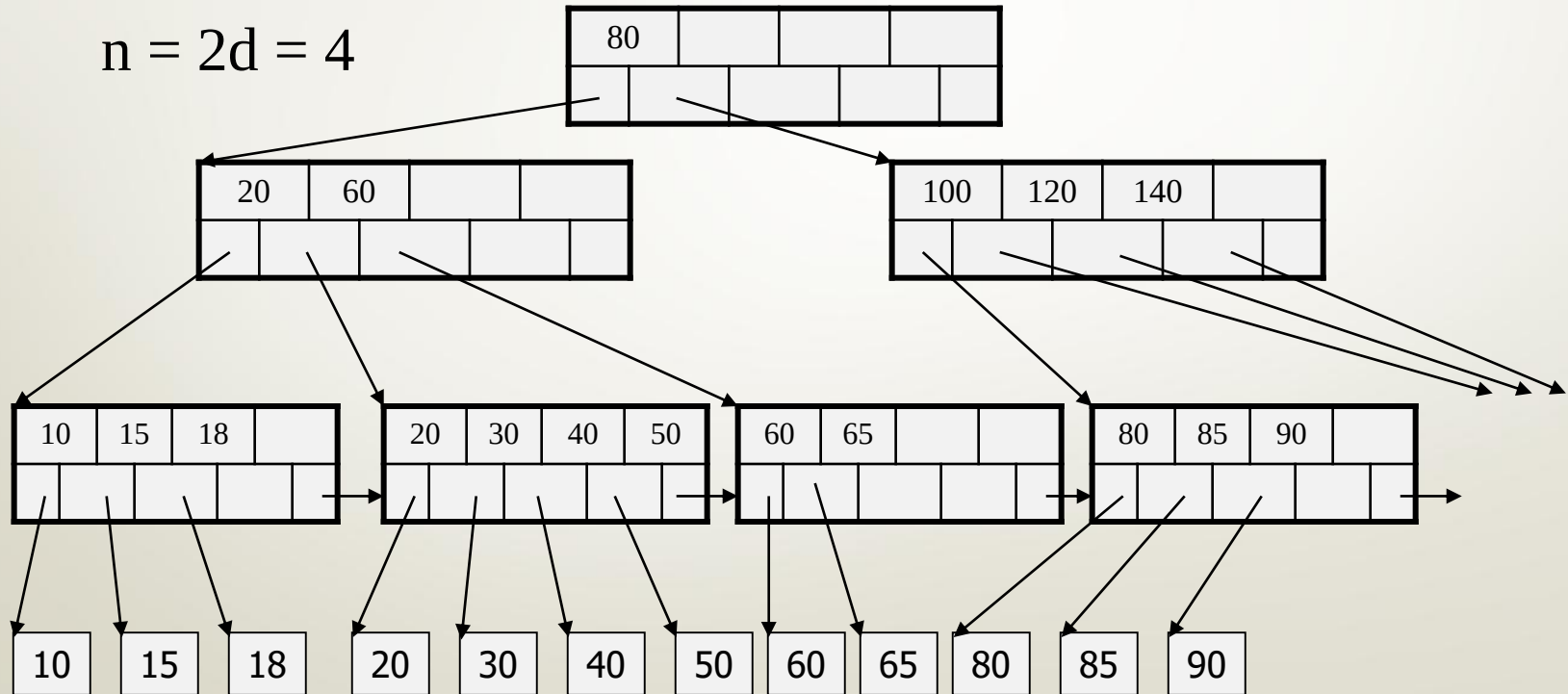
B+ Tree Example

Root can have 1 or more filled in keys

Rest have at least 2 and at most 4

$$d = 2$$

$$n = 2d = 4$$



B+ Tree Design

- How large should d be?
- Example:
 - Key size = 4 bytes
 - Pointer size = 8 bytes (e.g., (page #, record #) for leaf node pointers)
 - Block size = 4096 bytes
- $2d \times 4 + (2d+1) \times 8 \leq 4096$
- $d = 170; 2d = 340$

So up to 340 record ids (pointers) in leaf blocks

B+ Trees in Practice

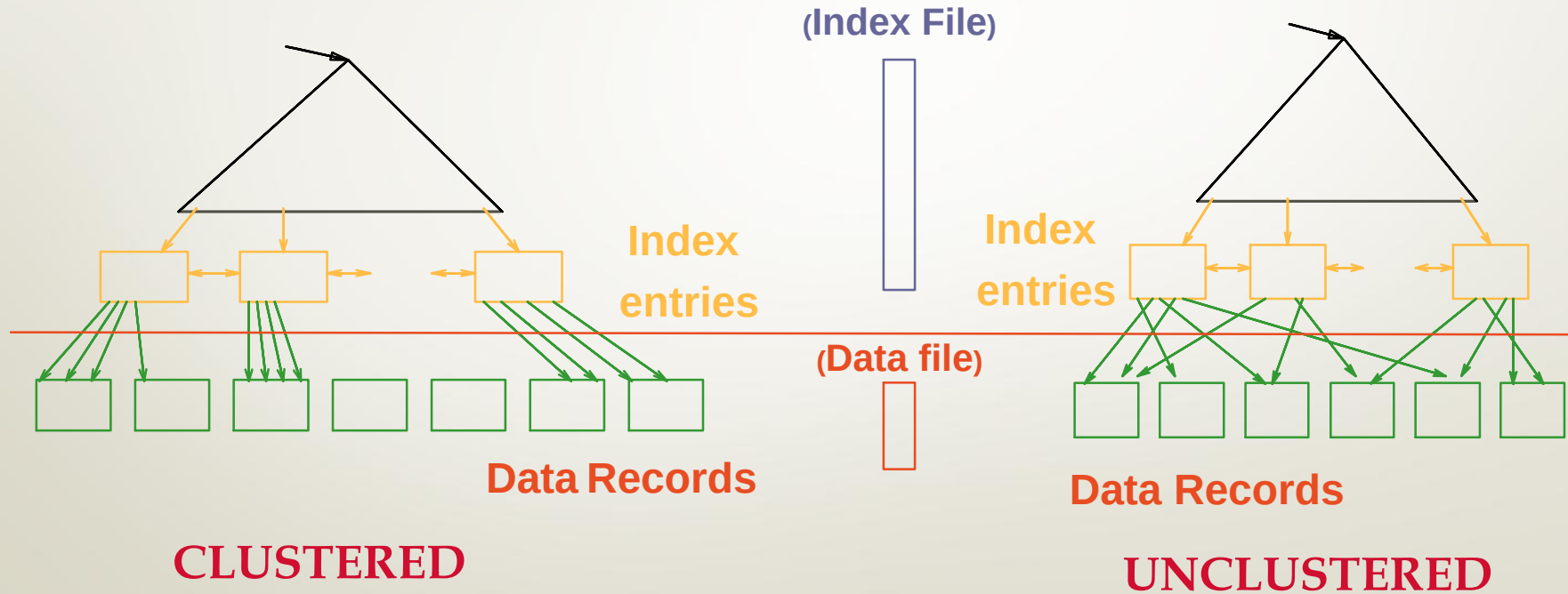
- Typical d : 100. Typical fill-factor: 66.5%.
 - average “fanout” = $66.5\% * 200 = 133$
- Typical capacities:
 - Height 3: $133^3 = 2,352,637$ records
 - Height 4: $133^4 = 312,900,700$ records
- Can often hold top levels in main memory:
 - Level 1 = 1 page = 8 Kbytes
 - Level 2 = 133 pages = 1 Mbyte
 - Level 3 = 17,689 pages = 133 MBytes

When Do B+ Trees Help?

- Types of queries to answer with a B+ Tree:
 - *Exact key value*, e.g., SELECT name FROM people WHERE age=20
 - *Range queries*, e.g., SELECT name FROM people WHERE age>=20 and age<=70
- Hash indexes are good only for exact-key-value search.

Question

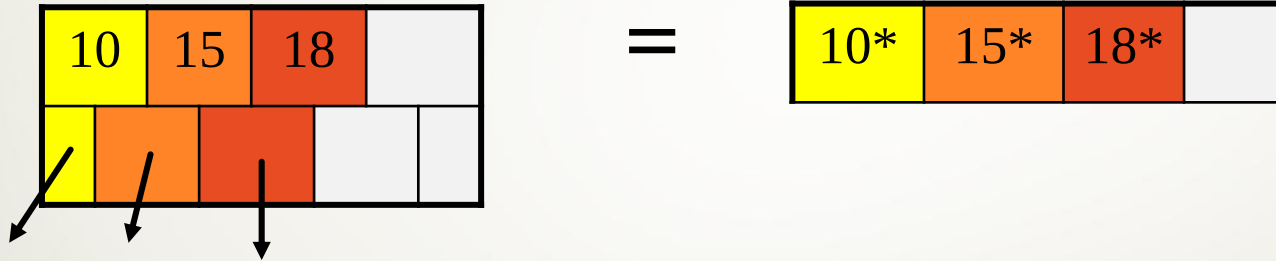
What type of queries that makes these index types different in performance?



Three alternatives for data entries

Data Entries

- Another notation for leaf nodes



- k^* is called the **data entry** for the search key value k
 - E.g., 10^* , 15^* are data entries for search key values 10, 15
- k^* may contain the search key value k and RID (pointer)
 - Other options exist, see next slide.

RID = Record id: location in file structure (e.g., (page #, record # inside the page))

Alternatives for Data Entry k^* in Index

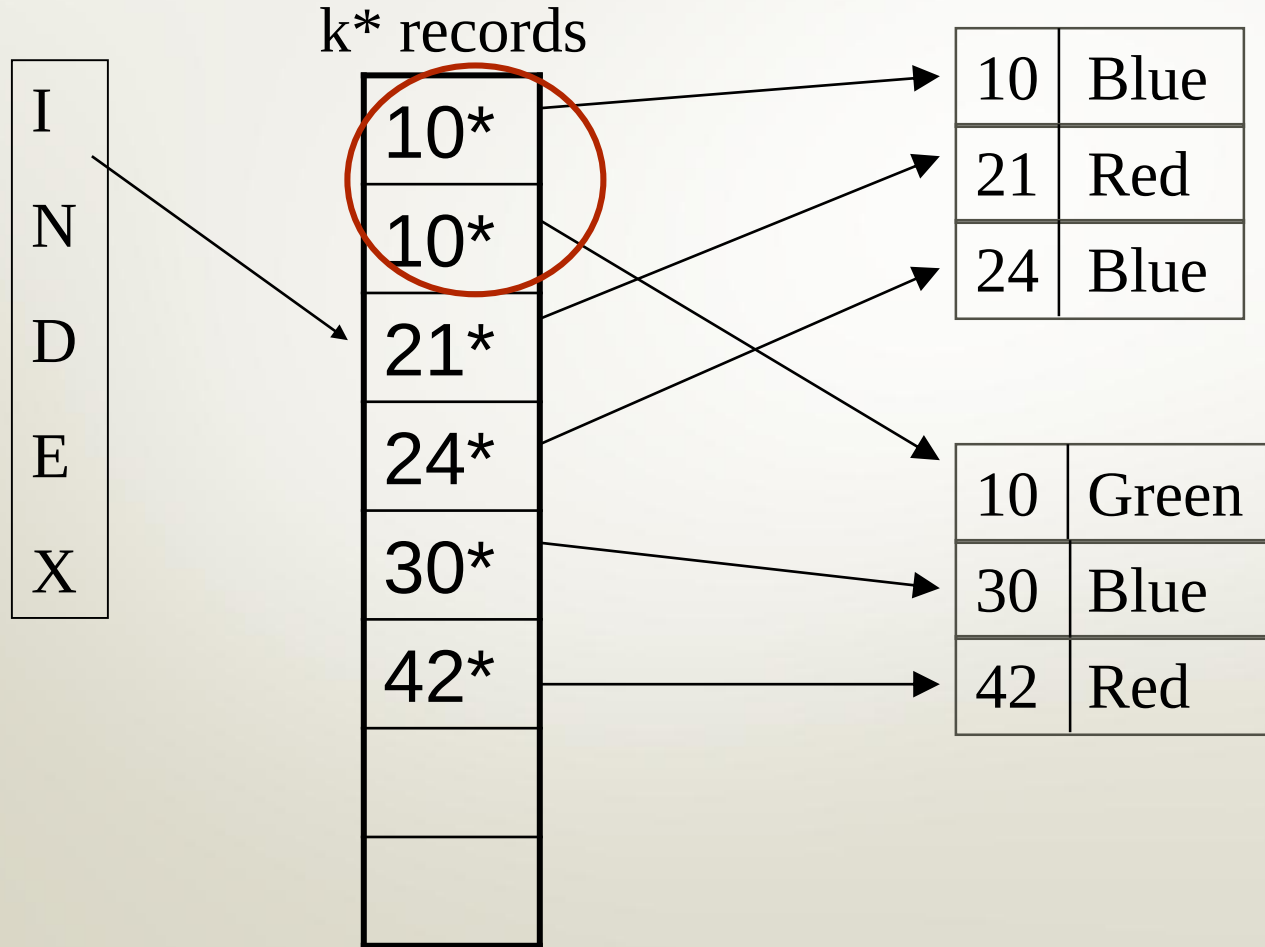
In a data entry k^* we can store:

1. Data record with key value k , or
2. $\langle k, \text{RID of data record with search key value } k \rangle$, or
3. $\langle k, \text{list of RIDs of data records with search key } k \rangle$

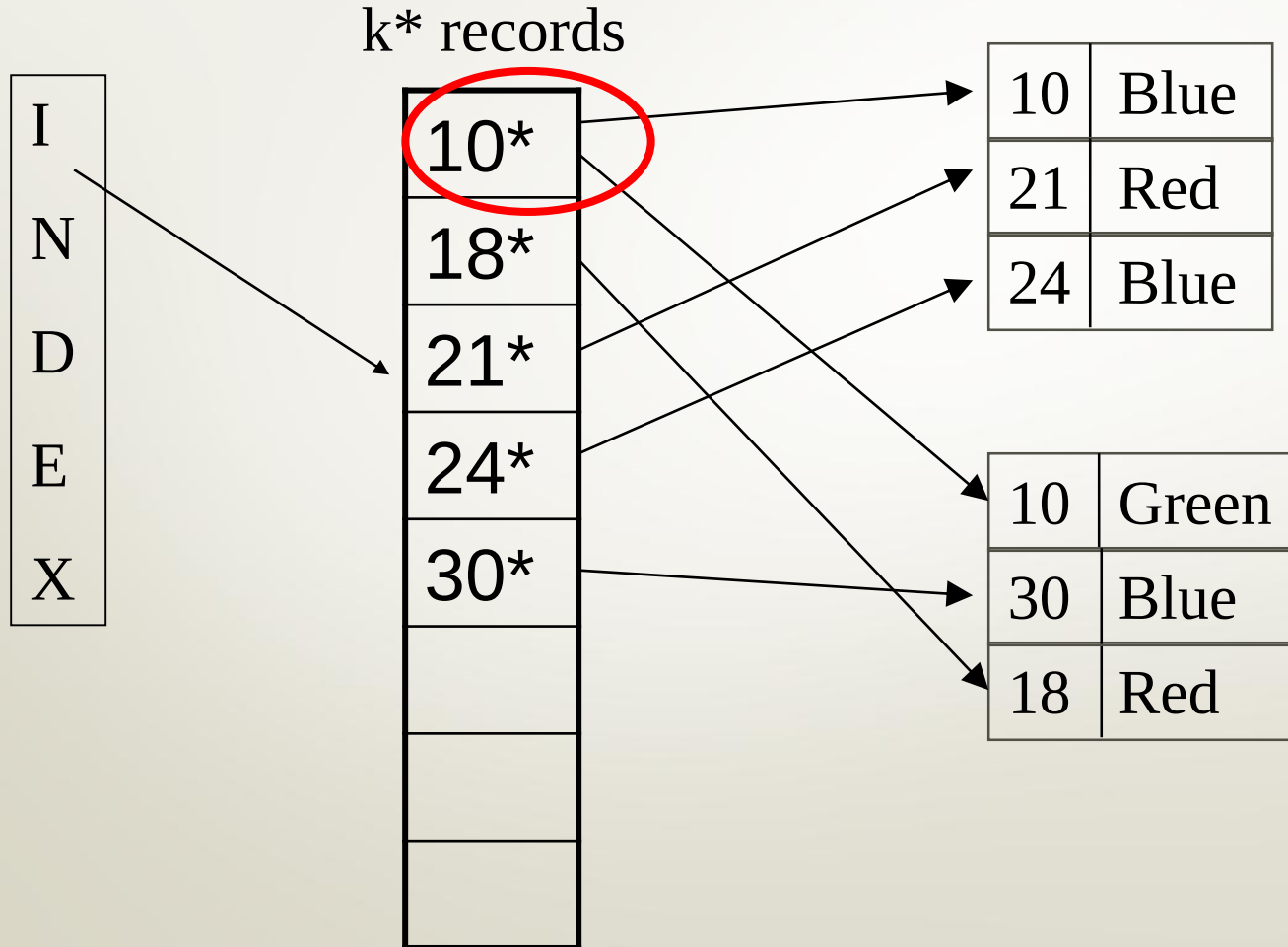
Choice of alternative for data entries is orthogonal to the indexing technique used to locate data entries with a given key value k .

- Examples of indexing techniques: B+ trees, hash-based structures

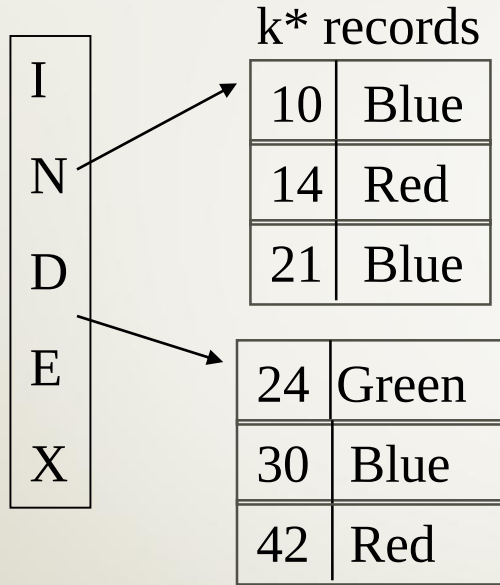
Alternative 2: One pointer (RID) per k*



Alternative 3: 1 or more pointers per k*



Alternative 1: k^* holds the data

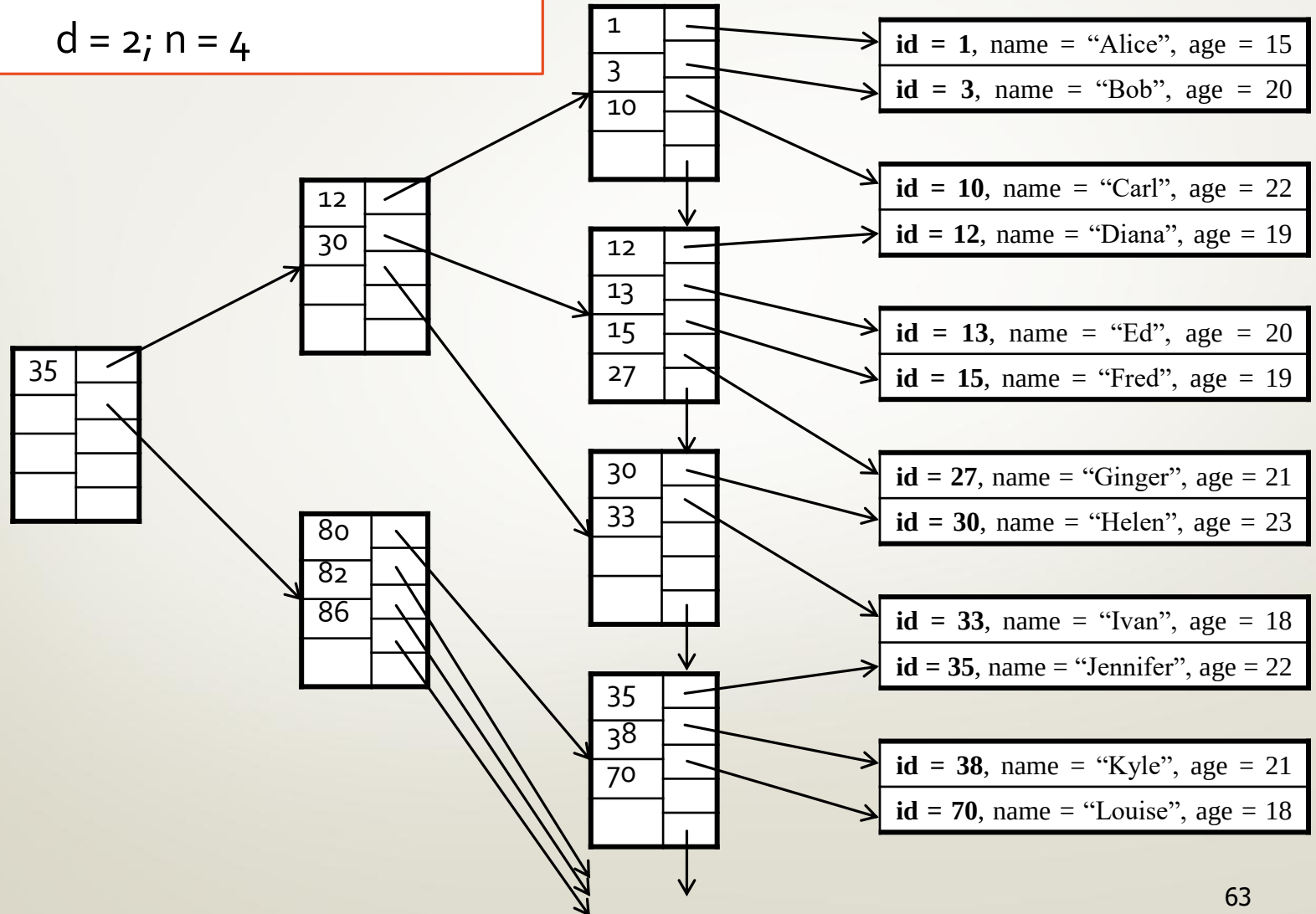


E.g., Leaf nodes of a B+tree index
or buckets in a hash index

Clustered dense (alternative two)

$d = 2; n = 4$

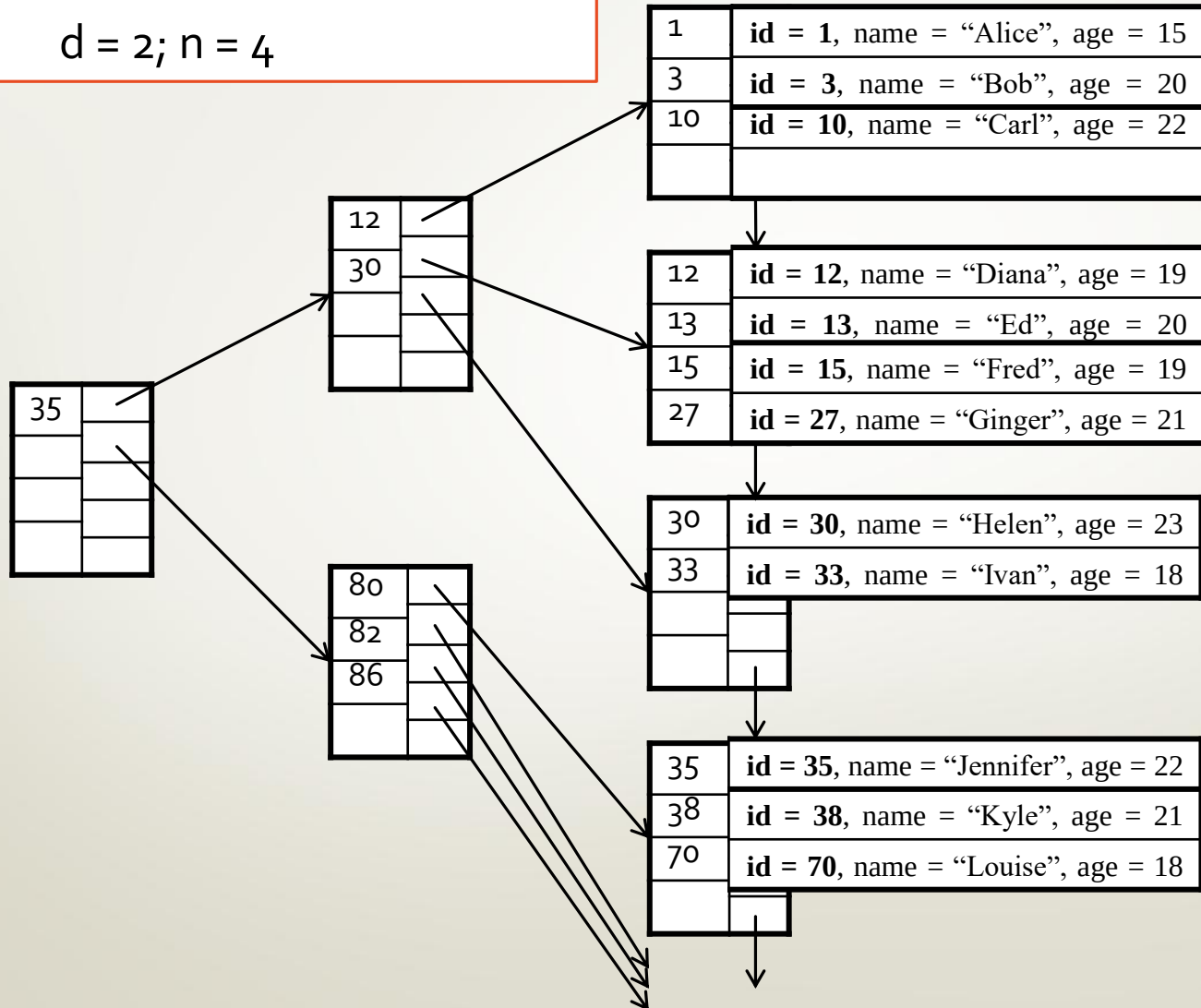
B+ Tree search key = **id**



Clustered dense (alternative one)

$d = 2; n = 4$

B+ Tree search key = **id**



Alternative 1

- **Alternative 1:**
 - If this is used, index structure is a file organization for data records.
 - There can be at most one single-attribute index on a given collection of data records using Alternative 1. Why?
 - If data records are very large, # of pages containing data entries is high. If there is one leaf pointer per page, this implies more leaf pointers and perhaps a deeper tree. So Alternative 1 is less attractive in this case.

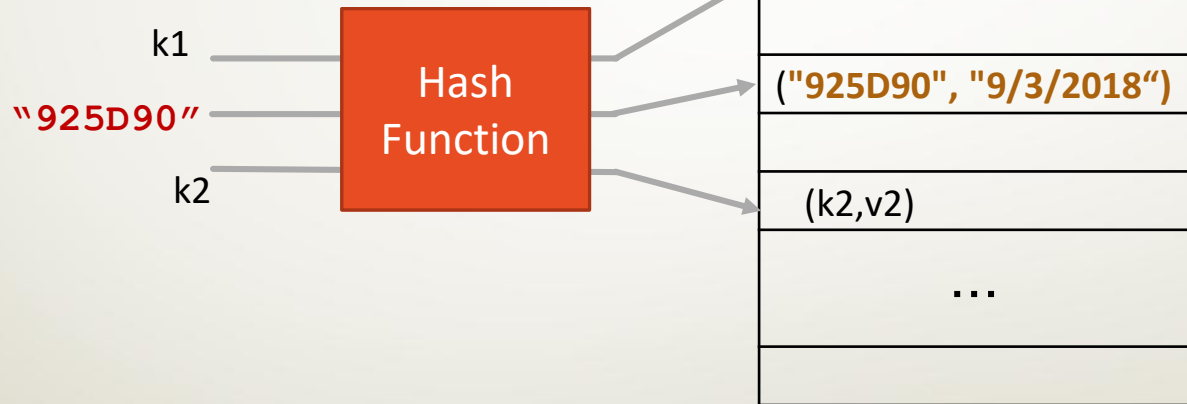
Alternatives for Data Entries (Contd.)

- Alternatives 2 and 3:
 - Alternative 3 is more compact than Alternative 2, but leads to variable-sized data entries even if search keys are of fixed length.

- ✓ B+tree index
- Hash index

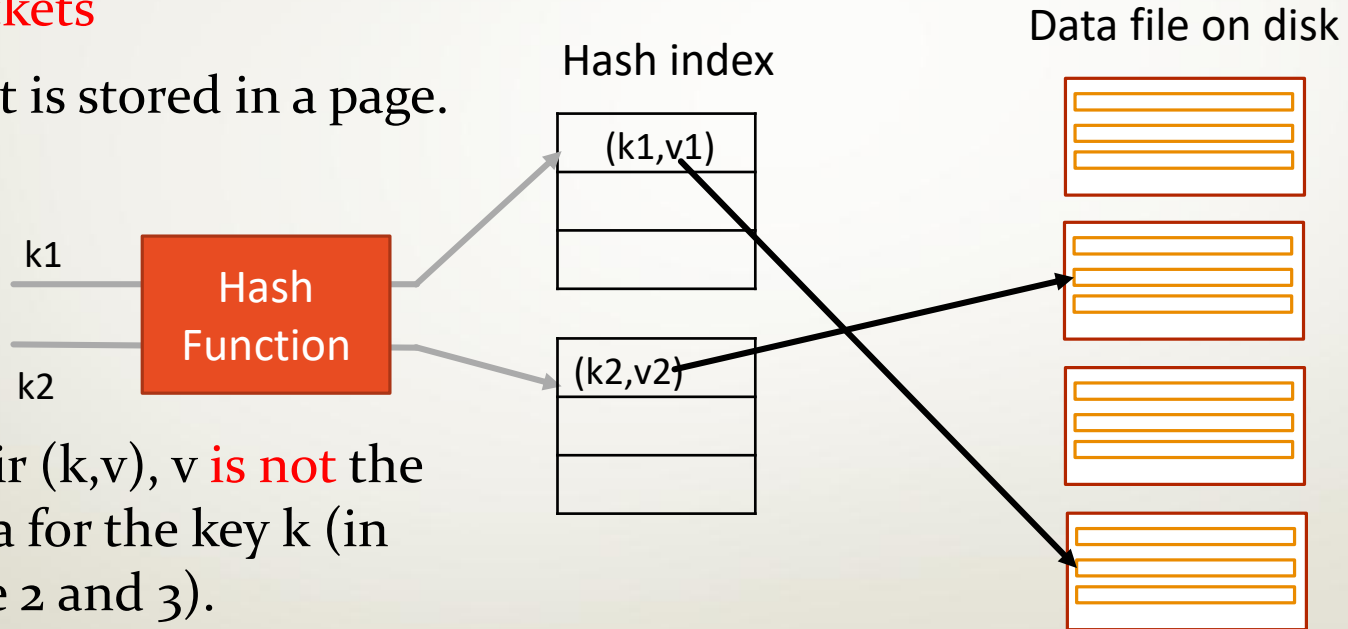
Hash Table - Review

Given a key k , a hash function h returns a **position** where (k,v) is located in the hash table (e.g., array)



Hash indexes in databases

- Hash table (index) is **split into buckets**
- A bucket is stored in a page.

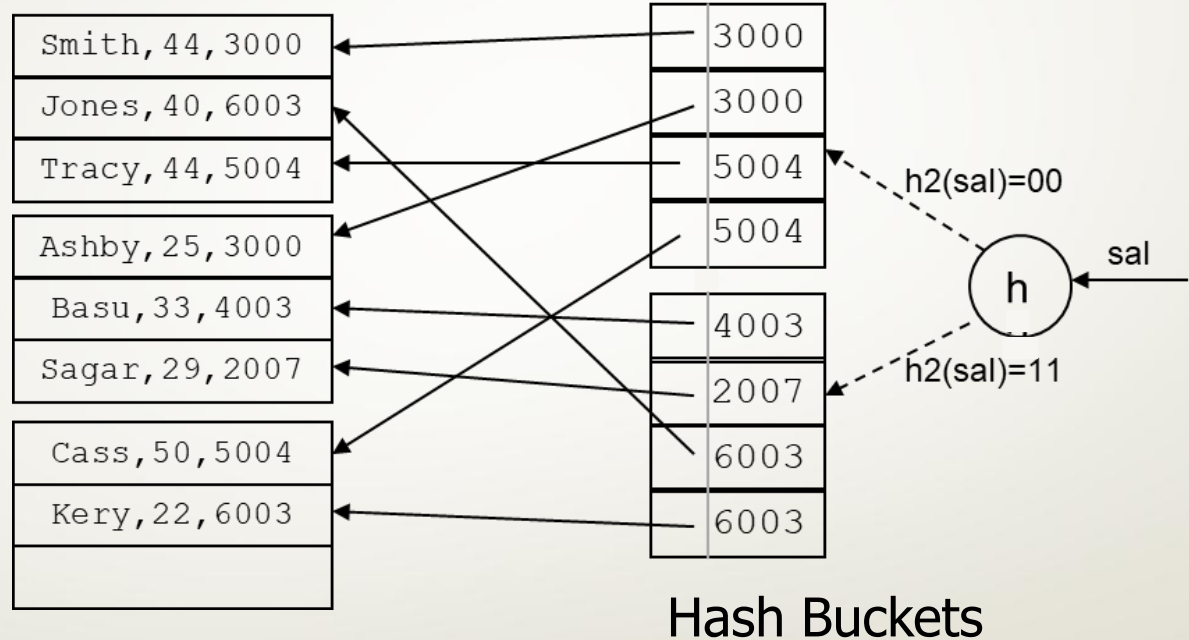


- For the pair (k, v) , v **is not** the actual data for the key k (in alternative 2 and 3).
- v is the **RID** for a record containing the search key k (assuming alternatives 2 or 3)

Hash-based Index Example

- Example h:
 $h(\text{sal}) = \text{sal} \bmod 4$
with four buckets:

00
01
10
11



Is this hash index
clustered or
unclustered?

Which one is larger (in
size) hash or B+tree
indexes?

Hash-Based Indexes

- Good for equality selections.
- Index is a collection of buckets.
 - Bucket = *primary page* plus zero or more *overflow pages*.
 - Buckets contain data entries.
- *Hashing function h* : $h(r)$ = bucket in which (data entry for) record r belongs. h looks at the *search key* fields of r .
 - *No need for “index entries” in this scheme.*
 - *i.e., no need for b+tree internal nodes.*

- ✓ B+tree index
- ✓ Hash index

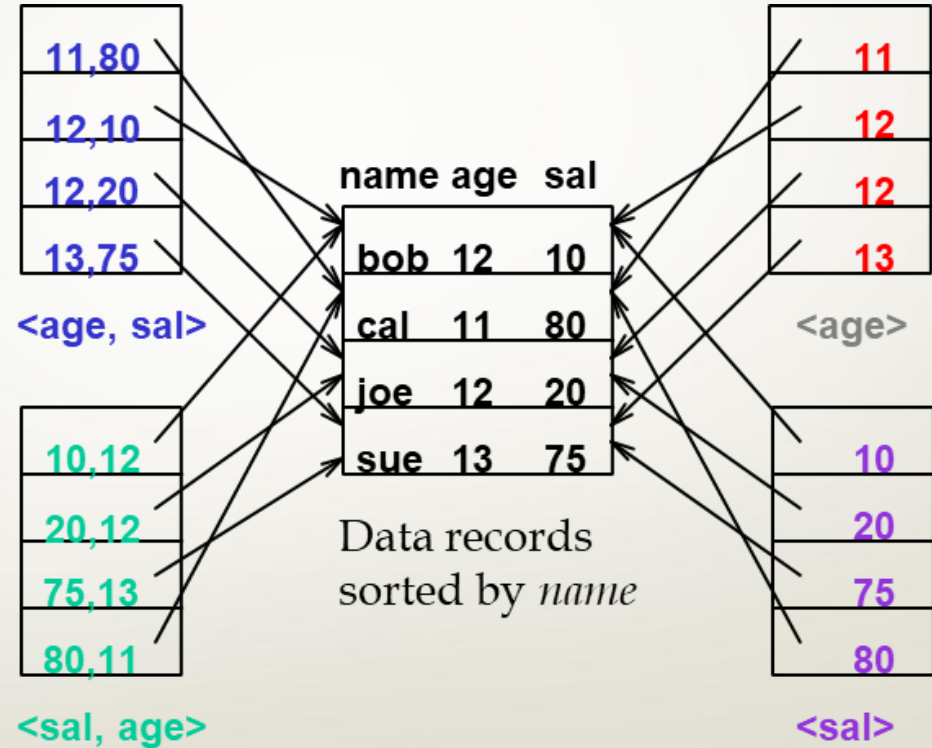
- Composite Search Keys
- Refinement for unclustered indexes

Using Composite Search Keys

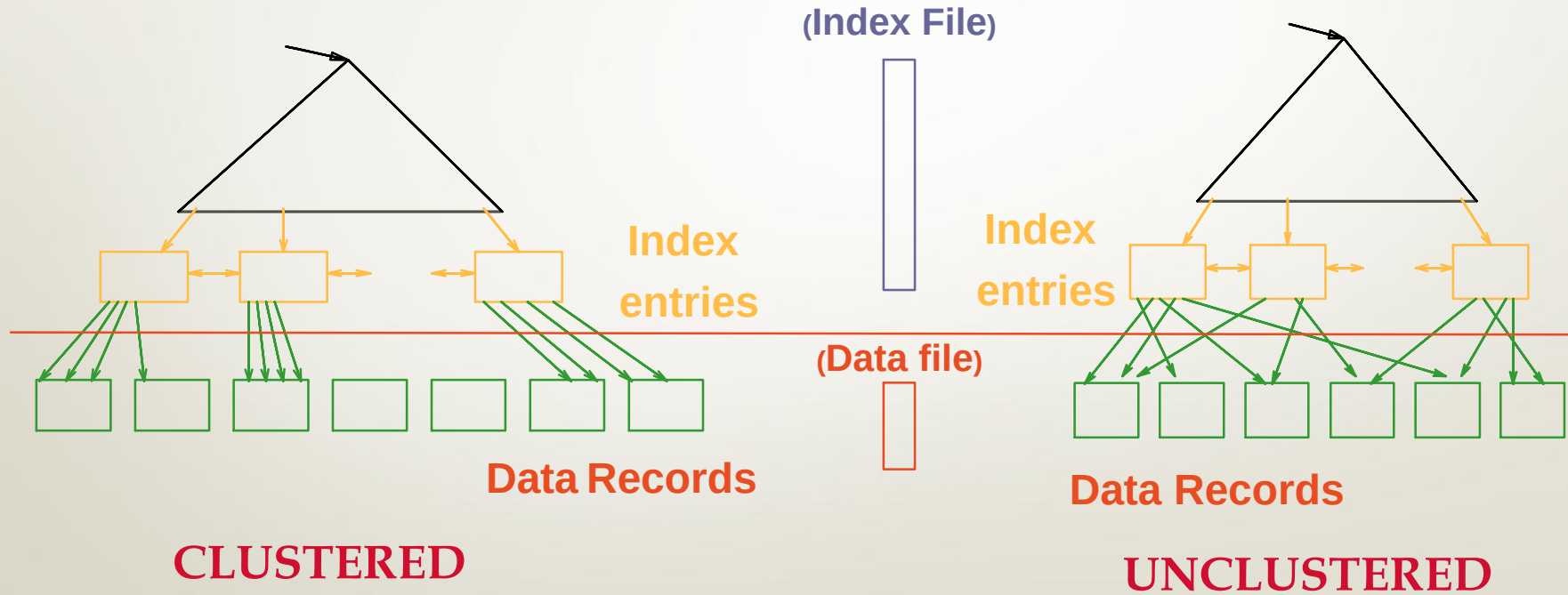
Which indexes are the most useful for each of these queries?

(most useful = the data entries are adjacent in the index
=> reading less index pages
=> less Page IOs
=> a faster query)

- Where age = 12
- Where age = 12 and sal = 20
- Where age = 12 and sal > 10
- Where age > 12 and sal > 30



Adjacent data entries

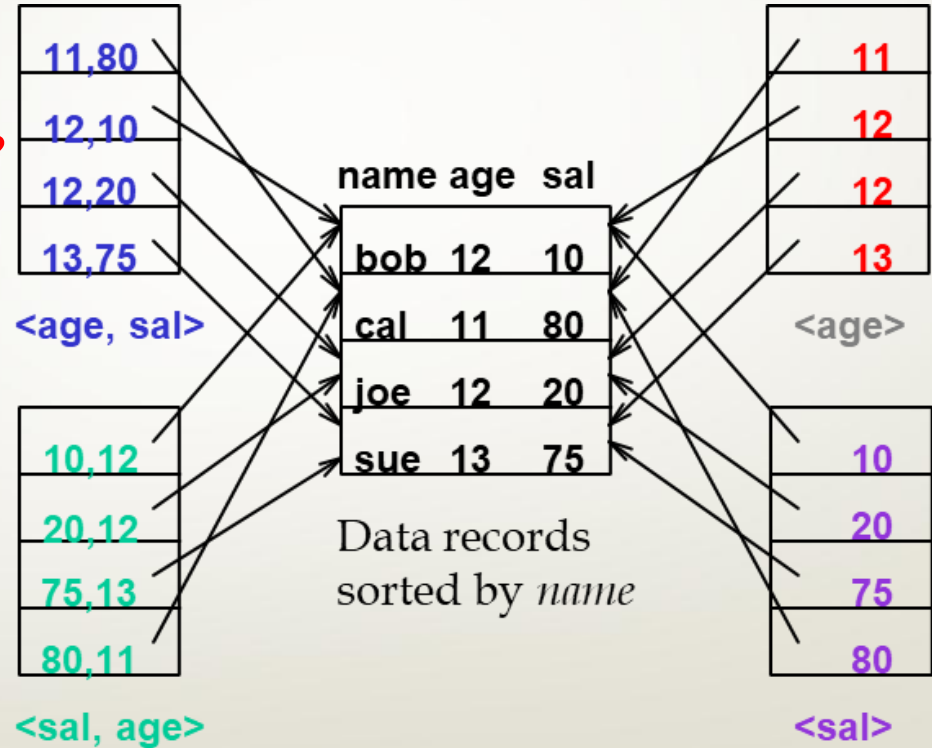


Using Composite Search Keys

Which indexes are the most useful for each of these queries?
(most useful = the data entries are adjacent in the index)

- age = 12
<age>
<age,sal>

other (less) useful indexes:
<sal, age>

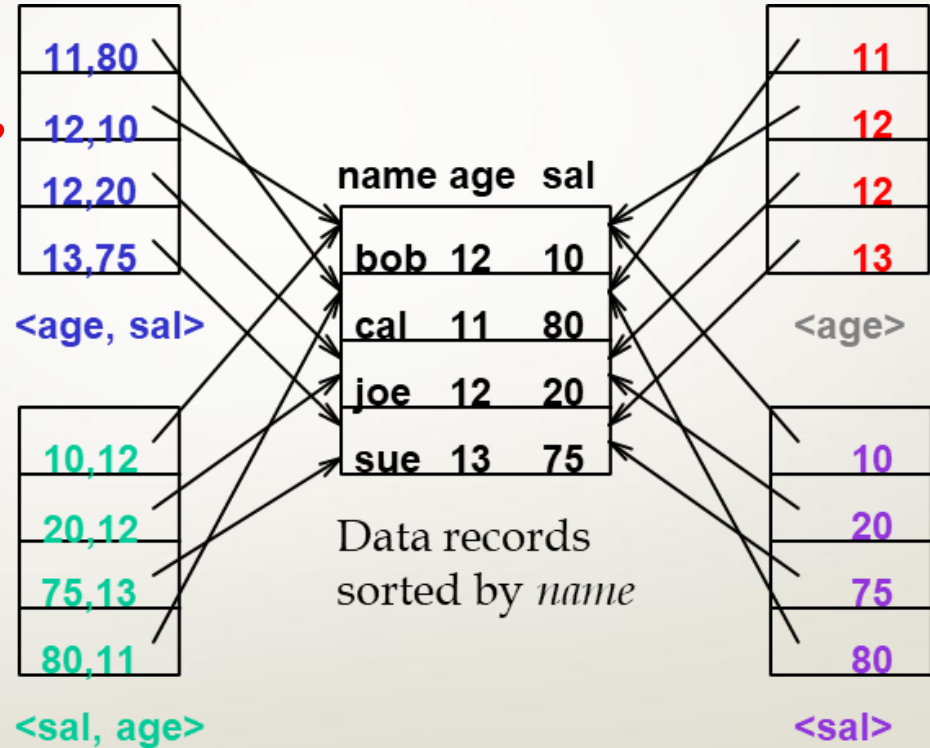


Using Composite Search Keys

Which indexes are the most useful for each of these queries?
(most useful = the data entries are adjacent in the index)

- age = 12 and sal = 20
 <age, sal>
 <sal, age>

other (less) useful indexes:
 <age>, <sal>, or both <age> and <sal>

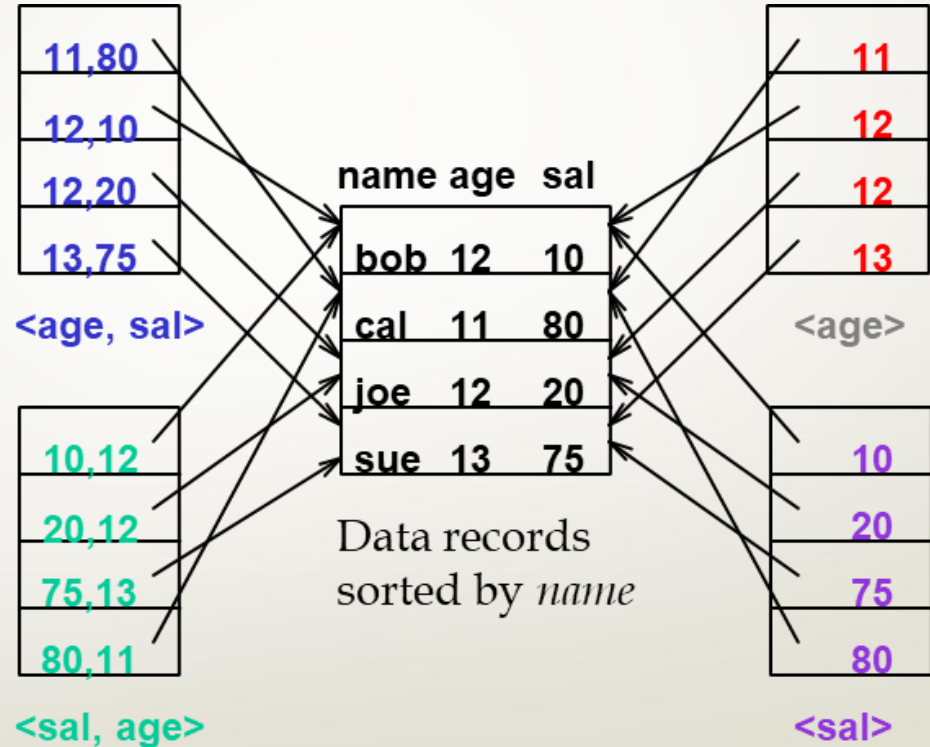


Using Composite Search Keys

- age = 12 and sal = 20

How to use both <age> and <sal> indexes?

- A = Find RIDs for age=12 in the **age index**
- B = Find RIDs for sal=20 in the **sal index**
- $C = A \cap B$
- Retrieve records whose RIDs are in C from the data file

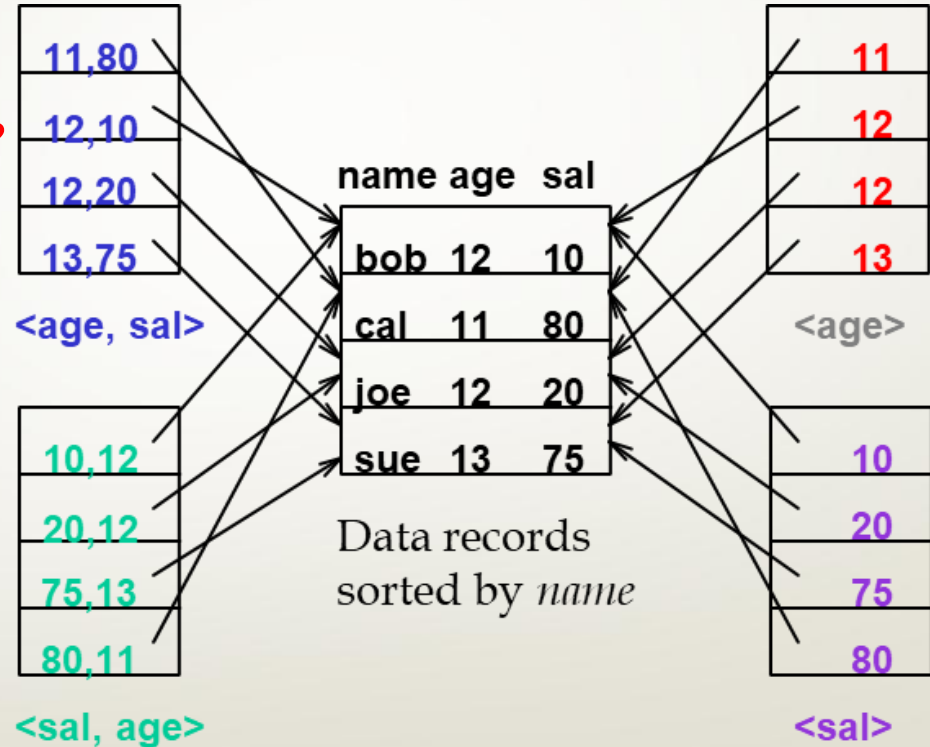


Using Composite Search Keys

Which indexes are the most useful for each of these queries?
(most useful = the data entries are adjacent in the index)

- age = 12 and sal > 10
best index for the query:
<age, sal>

other (less) useful indexes:
<sal, age>, <age>, <sal>, both
<age> and <sal>



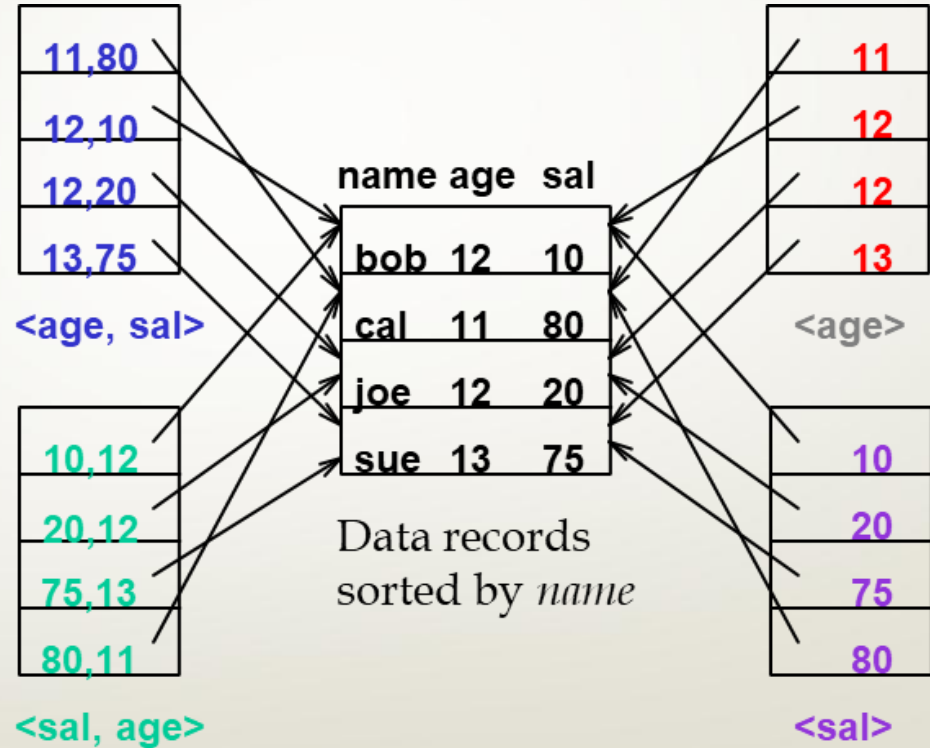
Using Composite Search Keys

Which indexes are the most useful for each of these queries?

(most useful = the data entries are adjacent in the index)

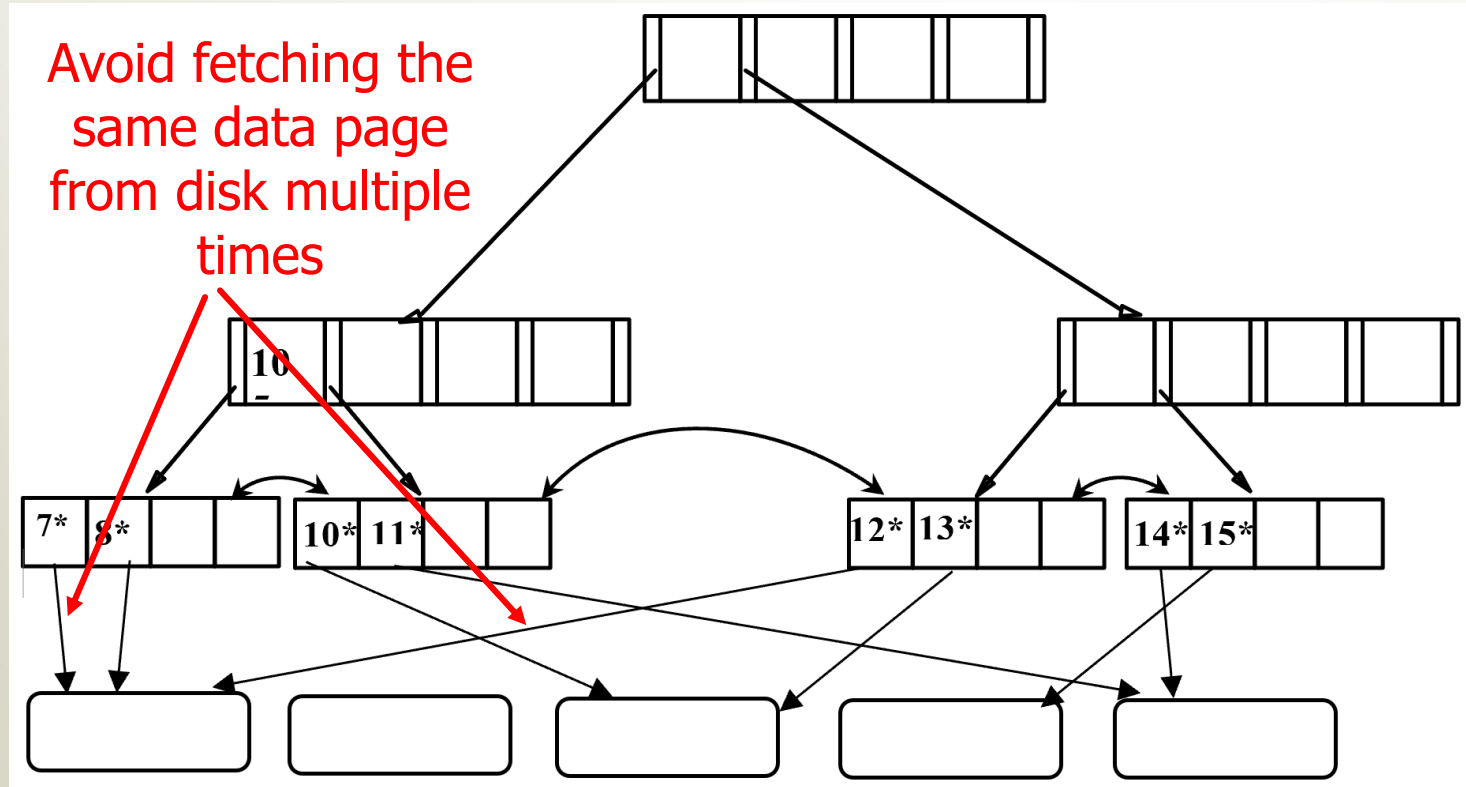
- age > 12 and sal > 30
no index has adjacent data entries.

other (less) useful indexes:
<age, sal>, <sal, age>, <age>,
<sal>, both <age> and <sal>



- ✓ Composite Search Keys
- Refinement for unclustered indexes

Important refinement for unclustered indexes



Important refinement for unclustered indexes

1. Find qualifying data entries.
2. **Sort the RIDs** of the data records to be retrieved.
3. **Fetch RIDs in order.**

This ensures that each data page is visited just once (though # of such pages likely to be higher than with clustering).

In homework and exam problems, you can assume a page is visited once in the case of unclustered indexes.

Exercise

Assume we have a table that includes student information:

Student(ID, username, name, Age)

Where ID is a key, username is also a key

There are four indexes on this table:

I₁: Unclustered hash index on <ID>

I₂: Clustered B+ tree on <username>

I₃: Unclustered B+ tree on <name>

I₄: Unclustered B+ tree on <Age>

For each of the following WHERE conditions, say which index you think is best to use and **why** you think it's the best.

Exercise

Question 1 :

Select * From Student

Where ID=905678123 AND Age > 18 and Age < 40;

- Use I1 (the unclustered hash index on ID).
- Fetch the record where ID = **905678123**, then check if the student age is between 18 and 40.
- The cost will be 1-2 page IOs (one for the hash bucket if the bucket is in disk and one for the data page containing the record with the required ID).
- We say that the condition ID=**905678123** is **selective**.
- Selective means that the condition is true for only a few rows/records in the data file.
- Selective conditions utilize indexes the most.

Exercise

Question 1 :

Select * From Student

Where ID=905678123 AND Age > 18 and Age < 40;

- Using I₄ (unclustered B+ tree on <Age>) would cost more page IOs.
 - The age condition is **not selective** (likely, we have many records with the required age range).
 - Also, the I₄ index is unclustered.

Exercise

Assume we have a table that includes student information:

Student(ID, username, name, Age)

Where ID is a key, username is also a key

There are four indexes on this table:

I₁: Unclustered hash index on <ID>

I₂: Clustered B+ tree on <username>

I₃: Unclustered B+ tree on <name>

I₄: Unclustered B+ tree on <Age>

Select * From Student

**Where (ID >= 1000 AND ID <=1020)
AND (name >= 'a' AND name < 'b') ;**

Assume both conditions returns the same number of students.

Which index do you think should be used for this query?

Exercise

Question 2:

```
Select * From Student
```

```
Where (ID >= 1000 AND ID <=1020)  
      AND (name >= 'a' AND name < 'b') ;
```

Assume that the number of students having an ID between 1000 and 1020 is the same as the number of students with a name that starts with the letter 'a'.

Use I₃ (unclustered B+ tree on <name>) as **B+tree are good for range queries** (remember the linked list in the leaf nodes).

After using the index to get the records with the required names, check the ID of those record.

Using I₁ (unclustered hash index on <ID>) is not as attractive. **Hash indexes are not that good in range queries.**

Using both I₃ and I₁ can be an option. Each index returns a set of RIDs. Retrieve (from disk) only records whose RID is in the two sets.

The cost of indexes

- **There is no such thing as a free lunch**
- What is the cost of indexes?
 - Space to store the index (index pages)
 - We need to update indexes in insert, update, and delete operations performed on the data file
- The more indexes you have, the slower your insert, update, and delete will be

Which attributes to index

- Pick a few indexes for each table
 - Create indexes for attributes included in the Where clause of the important queries (e.g., select password from student where username = 1234; is an important query as it is used in every log-in)
- Indexes on join attributes can be helpful!
 - Especially for the right relation in the join (the left relation is usually the smaller relation)
 - E.g., Select * From Student Join Grades on Student.ID = Grades.ID
 - Assuming student is the smaller table (on the left), then an index on Grades.ID can help speed up the join operation!

How to create an index (similar syntax in SQLite and MySQL)

- `CREATE INDEX idx_student_id ON Students (std_id);`
- `CREATE INDEX mg ON Students (major, gpa);`
- `CREATE UNIQUE INDEX idx_email ON Students (email);`
- `CREATE INDEX idx_gpa ON Students (gpa) Using Btree; // or hash (MySQL syntax)`
- Usually, there is a default index on the primary key (clustered index)
 - In some DBMSs, you may be able to change the clustered index to another attribute.

Summary

- Indexes speed up query execution
 - Avoid scanning the whole data file
 - Best scenario for index is selective queries especially in a large file (needle in a haystack)
- Cost of indexes (space and possibility of slow inserts/updates/deletes)
- Two types of indexes (other types exist)
 - B+tree indexes can support point and range queries
 - Hash indexes can support point queries
- Other properties (clustered, unclustered, dense, and sparse)